

функциональная страница

В курсовом проекте отражена разработка веб-приложения CinemaHome — домашней платформы для каталогизации и воспроизведения локального медиа-контента (фильмов и сериалов) на языке Java 21 с использованием фреймворка Spring Boot 3.4. Ключевой особенностью проекта является динамическая смена хранилища в рамках пользовательской сессии: система поддерживает два режима работы — реляционная СУБД Firebird (через Spring Data JPA / Hibernate) и файловое хранилище в формате JSON (через Jackson Databind). Переключение между источниками данных реализовано без перезапуска приложения и без изменения бизнес-логики.

В ходе работы спроектирована гексагональная (Port–Adapter) архитектура с разделением на слои Controller / Service / Port / Adapter / Repository, что обеспечивает слабую связанность модулей и лёгкую расширяемость. Реализована многоуровневая система безопасности с ролевой моделью (ADMIN, moderator, user) на базе Spring Security и алгоритма хеширования паролей BCrypt. Пользовательский интерфейс построен на серверном шаблонизаторе Thymeleaf с использованием встроенного HTML5-плеера для воспроизведения видеофайлов. Дополнительно реализован механизм рекомендаций на основе паттерна Observer (уведомления о релизах) и генератора настроения MoodAnalyzer.

Корректность работы ключевых модулей подтверждена модульными тестами (JUnit 5 + Mockito + AssertJ). Разработанный сервис представляет собой готовый инструмент для организации локального медиа-каталога и может быть использован в образовательных и домашних сценариях.

Табл. 8. Ил 15. Библ. 8

Содержание

Введение.....	5
1 Анализ технического задания.....	7
1.1 Цель и задачи разработки.....	7
1.2 Архитектура системы.....	12
1.3 Функциональные требования к системе.....	16
1.4 Управление медиаконтентом.....	19
2 Проектирование программы.....	24
2.1 Проектирование архитектуры и структуры данных системы.....	24
2.2 Проектирование информационной модели и базы данных.....	33
3 Разработка приложения.....	35
3.1 Выбор и обоснование технологического стека.....	35
3.2 Реализация системы адаптеров хранилища (Port–Adapter).....	38
3.3 Реализация работы с мультимедиа-контентом.....	49
3.4 Реализация управления ролями и безопасности.....	56
3.5 Реализация пользовательского интерфейса.....	60
3.6 Реализация модуля рекомендаций и уведомлений.....	65
3.7 Архитектурные особенности реализации.....	70
4 Тестирование.....	74
4.1 Цели и стратегия тестирования.....	74
4.2 Ручное (функциональное) тестирование.....	75
4.3 Инструментарий автоматизированного тестирования.....	84
Заключение.....	91
Список использованных источников.....	93

					МИВУ.09.03.02 009 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Медведев С. В			Курсовой проект по теме: «Создание адаптивной платформы локального медиаконтента с динамической сменой хранилища»	Лит.	Лист	Листов
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								

функциональная страница

					МИВУ.09.03.02 009 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Курсовой проект по теме: «Создание адаптивной платформы локального медиаконтента с динамической сменой хранилища»	Лит.	Лист	Листов
Разраб.		Медведев С. В						
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								

функциональная страница

					МИВУ.09.03.02 009 ПЗ	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

ВВЕДЕНИЕ

С развитием цифровых технологий и повсеместным распространением локальных медиатек всё более востребованными становятся инструменты для каталогизации, поиска и воспроизведения аудио- и видеофайлов в пределах одной домашней сети или персонального компьютера. Современные веб-технологии позволяют создавать интерактивные сервисы, которые делают процесс работы с медиаконтентом удобным и эстетически привлекательным [1].

Тема курсового проекта — создание адаптивной платформы локального медиаконтента с динамической сменой хранилища. Объектом исследования является процесс организации, хранения и воспроизведения локальной видеотеки. Предметом исследования — разработанное веб-приложение CinemaHome, реализующее данный функционал.

Цель курсового проекта — спроектировать и разработать многофункциональное веб-приложение для управления каталогом фильмов и сериалов с возможностью воспроизведения видео через встроенный HTML5-плеер и с поддержкой двух независимых механизмов хранения данных (СУБД и JSON-файлы).

Исходя из поставленной цели, определены следующие задачи:

- ~ изучение литературных источников по теме проектирования баз данных, REST-архитектуры и разработки веб-приложений;

- ~ проектирование структуры реляционной базы данных для хранения информации о фильмах, сериалах, сезонах, эпизодах, актёрах, жанрах, режиссёрах и пользователях.

- реализация серверной части приложения (backend) с использованием паттернов проектирования (Port-Adapter, Strategy, Observer);

- создание пользовательского интерфейса (frontend) на базе Thymeleaf;

- реализация системы ролей и безопасности;

- тестирование и отладка разработанного сервиса.

Результатом работы является готовое веб-приложение, запускаемое на <http://localhost:8080>, предоставляющее каталог, плеер, административную панель и возможность переключения между режимами хранения данных.

					МИВУ.09.03.02 009 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

1 Анализ технического задания

1.1 Цель и задачи разработки

Современная медиасреда характеризуется экспоненциальным ростом объемов цифрового контента. По данным исследования Cisco Visual Networking Index, к 2026 году видео будет составлять более 82 % всего интернет-трафика, а локальные медиатеки пользователей в среднем содержат от 200 до 2000 единиц контента [1]. В этих условиях традиционные файловые менеджеры и проводники операционных систем перестают удовлетворять потребностям пользователей: они не предоставляют ни удобной каталогизации, ни контекстного поиска, ни персонализированных рекомендаций.

Особую актуальность задача приобретает в контексте цифрового суверенитета и импортозамещения. Большинство популярных медиасерверов (Plex, Emby) являются проприетарными решениями с закрытым исходным кодом, хранение метаданных в которых происходит на зарубежных серверах. Это создаёт риски как в плане конфиденциальности данных, так и в плане доступности сервиса при изменении геополитической обстановки. Разработка отечественного решения с открытым исходным кодом и локальным хранением данных становится не просто учебной, но и практически значимой задачей.

Дополнительным фактором актуальности является потребность в гибких механизмах хранения. В реальных условиях эксплуатации нередки ситуации, когда развёртывание полноценной СУБД нецелесообразно (маломощное оборудование, однопользовательский режим, демонстрационные сценарии), но при этом требуется сохранить возможность миграции на промышленное хранилище без переписывания бизнес-логики. Данная задача напрямую соотносится с принципами гексагональной архитектуры (Ports & Adapters), предложенной Алистером Коборном в 2005 году [2].

					МИВУ.09.03.02 009 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

Целью курсового проекта является проектирование и разработка многофункционального веб-приложения CinemaHome для управления локальным каталогом фильмов и сериалов с возможностью воспроизведения видео через встроенный HTML5-плеер и с поддержкой двух независимых механизмов хранения данных — реляционной СУБД Firebird SQL и файлового хранилища в формате JSON.

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести анализ существующих решений (аналогов) на рынке с целью выявления сильных и слабых сторон, определения ключевых функциональных требований и выявления ниши для разрабатываемого продукта;
- Выбрать оптимальный стек технологий и инструментов для разработки, обосновав выбор каждой технологии с учётом требований к производительности, безопасности и сопровождаемости;
- Спроектировать структуру реляционной базы данных для хранения информации о восьми типах сущностей: фильмах, сериалах, сезонах, эпизодах, актёрах, жанрах, режиссёрах и пользователях;
- Разработать серверную часть приложения (backend) с использованием современных паттернов проектирования (Port-Adapter, Strategy, Observer), обеспечив слабую связанность модулей и лёгкую расширяемость;
- Создать пользовательский интерфейс (frontend) на базе серверного шаблонизатора Thymeleaf для трёх основных ролей: администратора, модератора и обычного пользователя;
- Реализовать модуль динамической смены хранилища (БД ↔ JSON), позволяющий переключаться между источниками данных в рамках пользовательской сессии без перезапуска приложения;
- Реализовать многоуровневую систему безопасности с ролевой моделью доступа и хешированием паролей по алгоритму Bcrypt;
- Провести модульное (Unit Testing) и ручное функциональное тестирование готового приложения, подтвердив корректность работы всех ключевых подсистем.

В ходе исследования были рассмотрены три наиболее популярных решения для управления медиатеками: Plex Media Server, Jellyfin и Emby. Каждый из них обладает уникальными характеристиками, определяющими его целевую аудиторию и сценарии использования.

Plex Media Server — коммерческий продукт с закрытым исходным кодом, предоставляющий широкий спектр функций: автоматическое получение метаданных из онлайн-источников (TMDB, TVDB), поддержку аппаратного транскодирования, мобильные приложения для всех популярных платформ. Однако его ключевым недостатком является обязательная привязка к облачным серверам Plex для аутентификации и синхронизации метаданных. Это создаёт зависимость от стороннего сервиса и ограничивает использование в автономных средах. Кроме того, ряд продвинутых функций (офлайн-загрузки, синхронизация между устройствами, пропуск интро) доступны только в платной подписке Plex Pass (от \$4.99/месяц) [4].

Jellyfin — полностью бесплатный форк проекта Emby с открытым исходным кодом (лицензия GPL-2.0). Он не требует подключения к внешним серверам и предоставляет весь функционал без ограничений. В качестве хранилища метаданных используется встроенная СУБД SQLite, что делает его легковесным, но ограничивает возможности масштабирования. Jellyfin активно развивается сообществом и имеет хорошую расширяемость через плагины, однако его архитектура не предполагает динамической смены хранилища — все данные жёстко привязаны к SQLite [5].

Emby — проприетарный продукт, сочетающий черты Plex и Jellyfin. Он предоставляет локальный сервер с возможностью автономной работы, но для полного раскрытия функционала (аппаратное транскодирование, мобильные приложения) требуется платная подписка Emby Premiere. Хранение данных также реализовано через SQLite, без возможности переключения на альтернативные источники.

В ходе исследования были рассмотрены популярные решения для управления медиатеками: Plex Media Server, Jellyfin и Emby. Анализ аналогов и обоснование выбора технологий представлены в таблицах 1 и 2.

Таблица 1 — Сравнительный анализ аналогов и разрабатываемого решения

Критерий сравнения	Plex Media Server	Jellyfin	Emby	Разрабатываемое приложение
Среда развертывания	Облако + локально	Облако (SaaS)	Локальный сервер	Локальный сервер
Хранение данных	Проприетарная БД	SQLite	SQLite	Firebird SQL / JSON
Исходный код	Закрытый	Открытый (GPL-2.0)	Закрытый	Открытый
Стоимость использования	Freemium	Бесплатно	Freemium	Бесплатно
Динамическая смена хранилища	Отсутствует	Отсутствует	Отсутствует	Реализована (DB / JSON)
Автономная работа	Ограничена	Полная	Частичная	Полная
Интеграция со своей БД	Только через API	Через плагины	Через REST API	Прямая работа (JPA/Jackson)
Транскодирование видео	Есть (в т.ч. аппаратное)	Есть (FFmpeg)	Есть (платно)	Отсутствует (не требуется)
Ролевая модель	2 роли (admin/user)	2 роли	3 роли	3 роли (ADMIN/moderator/user)

Таблица 2 — Обоснование выбора технологий

Технология	Версия	Назначение	Аналоги	Почему выбрана
Java	21	Основной язык программирования	C#, Python, PHP, Kotlin	Современный LTS-релиз с долгосрочной поддержкой до 2031 г. Доступ к новым языковым конструкциям: record patterns, pattern matching for switch, virtual threads (Project Loom). Строгая типизация обеспечивает надежность на этапе компиляции.
Spring Boot	3.4.0	Фреймворк для backend-разработки	Quarkus, Micronaut, Django, Express	Де-факто стандарт Enterprise-разработки на Java. Готовые стартеры Spring Data JPA, Spring Security, Spring Web позволяют сократить время разработки в 3–5 раз. Полная поддержка Jakarta EE 10.

Firebird SQL	3.0+	Реляционная СУБД	MySQL, PostgreSQL, SQLite	Легковесная (установочный пакет ~15 МБ), кроссплатформенная, не требует выделенного серверного процесса в embedded-режиме. Идеально подходит для локальных и учебных проектов. Поддержка BLOB-полей для хранения описаний.
Hibernate	6.6.2.Final	ORM-фреймворк	EclipseLink, MyBatis	Реализация спецификации JPA 3.1. Автоматическое построение SQL-запросов, кеш первого и второго уровня, поддержка сложных маппингов (ManyToMany, OneToMany). Community Dialects обеспечивает работу с Firebird.
Jackson Databind	2.17.0	Работа с JSON	Gson, Moshi	Стандарт де-факто в экосистеме Spring. Глубокая интеграция со Spring Boot, мощные возможности кастомной сериализации/десериализации, поддержка полиморфных типов.
Thymeleaf	3.1+	Серверный шаблонизатор	React, Angular, Vue.js	Рендеринг HTML на стороне сервера (SSR) избавляет от необходимости писать отдельный REST API. «Натуральные» шаблоны (валидный HTML даже без серверной обработки). Тесная интеграция со Spring через Spring EL.
Spring Security	6.x	Безопасность и аутентификация	Apache Shiro, Keycloak	Глубокая интеграция со Spring Boot, поддержка современных стандартов (OAuth 2.0, OIDC), гибкая ролевая модель, защита от CSRF/XSS/Clickjacking «из коробки».
BCrypt	через Spring Security	Хеширование паролей	Argon2, PBKDF2	Адаптивная функция хеширования с настраиваемым коэффициентом стоимости. Устойчива к атакам по радужным таблицам и брутфорсу на GPU.
Gradle	8.5	Система сборки	Maven, Ant	Гибкий DSL на основе Groovy, инкрементальная сборка, встроенный dependency management, официальная поддержка Spring Boot plugin.
JUnit 5 + Mockito + AssertJ	5.x / 5.x / 3.x	Модульное тестирование	TestNG, Spock	Современный стандарт тестирования в Java-экосистеме. Поддержка параметризованных тестов, fluent-assertions, мокирования зависимостей.

springdoc-openapi	2.3.0	API-документация	Springfox (устарел)	Автоматическая генерация OpenAPI 3.0 спецификации и Swagger UI из аннотаций. Активно поддерживается, в отличие от заброшенного Springfox.
Lombok	1.18.36	Сокращение boilerplate-кода	Record types (Java 14+)	Автоматическая генерация геттеров, сеттеров, конструкторов, equals/hashCode через аннотации (@Data, @AllArgsConstructor). Существенно сокращает объём шаблонного кода.

Таким образом, выбранный технологический стек является сбалансированным: он сочетает зрелые промышленные технологии (Spring Boot, Hibernate, Spring Security) с легковесными решениями (Firebird, Thymeleaf), что делает приложение одновременно надёжным и простым в развёртывании.

1.2 Архитектура системы

Архитектура программного обеспечения — это фундаментальная организация системы, воплощённая в её компонентах, их отношениях друг к другу и к среде, а также принципы, направляющие её проектирование и эволюцию [3]. Выбор архитектуры определяет такие нефункциональные характеристики системы, как масштабируемость, сопровождаемость, тестируемость и расширяемость.

В современной практике веб-разработки выделяют несколько основных архитектурных паттернов:

- MVC (Model-View-Controller) — классический паттерн разделения ответственности, широко применяемый в Spring MVC. Недостаток: жёсткая связь между бизнес-логикой и механизмами хранения данных;
- Layered Architecture (слоёная архитектура) — разделение на Presentation, Business, Persistence, Database слои. Обеспечивает чёткую иерархию, но не решает проблему зависимости бизнес-логики от конкретных реализаций;
- Clean Architecture (чистая архитектура) Роберта Мартина — концентрическая модель, где бизнес-правила находятся в центре и не зависят от внешних слоёв;

– Hexagonal Architecture (гексагональная архитектура, Ports & Adapters)
Алистер Коуборн — развитие идей Clean Architecture с акцентом на инверсию зависимостей через порты и адаптеры [2].

Для проекта CinemaHome была выбрана именно гексагональная архитектура, поскольку она наилучшим образом решает ключевую задачу проекта — динамическую смену хранилища данных. В этой архитектуре бизнес-логика зависит только от абстрактных интерфейсов (портов), а конкретные реализации (адаптеры) подключаются через механизм Dependency Injection.

Port (порт) — Java-интерфейс, определяющий контракт доступа к данным. В проекте определены 8 портов: MoviePort, SeriesPort, ActorPort, GenrePort, DirectorPort, EpisodePort, SeasonPort, UserPort. Каждый порт описывает минимально необходимый набор операций (CRUD), достаточный для бизнес-логики, но не раскрывающий детали реализации.

Adapter (адаптер) — конкретная реализация порта. Для каждого порта существуют две параллельные реализации:

- Db*Adapter — работа через Spring Data JPA с СУБД Firebird;
- Json*Adapter — работа через Jackson Databind с файлами *.json в директории /home/student/cinema-json/.

Обе реализации имеют идентичный контракт, что позволяет взаимозаменять их без изменения вызывающего кода.

Strategy (стратегия) — поведенческий паттерн, позволяющий выбирать алгоритм в runtime. В проекте он реализован через класс DataModeService, который хранит выбранный режим (DB или JSON) в HttpSession и предоставляет сервисам метод currentPort(), возвращающий нужный адаптер:

```
private MoviePort currentPort() {  
    DataMode mode = dataModeService.getMode();  
    return (mode == DataMode.JSON) ? jsonMovieAdapter :  
    dbMovieAdapter;  
}
```

Это решение обеспечивает соблюдение принципа Open/Closed Principle из SOLID: добавление нового хранилища (например, MongoDB или PostgreSQL) требует лишь создания нового класса-адаптера и регистрации его в Spring-контексте — бизнес-логика останется неизменной.

Observer (наблюдатель) — поведенческий паттерн, реализующий механизм подписки на события. В проекте он используется для системы уведомлений о релизах. Интерфейс ReleaseNotifier имеет три реализации:

- EmailNotifier (@Primary) — имитация отправки email;
- InAppNotifier — уведомление внутри приложения;
- MoodNotifier — персонализированное уведомление с учётом настроения пользователя.

Сервис ReleaseNotifierService получает список всех наблюдателей через @Autowired List<ReleaseNotifier> и обходит его, вызывая sendNotification() у каждого. Добавление нового канала уведомлений (например, Telegram-бот) требует лишь создания нового класса, реализующего интерфейс ReleaseNotifier.

Layered Architecture (слоёная архитектура) — общее разделение приложения на 5 логических слоёв, каждый из которых имеет строго определённую ответственность:

- Controller Layer (@Controller, @RestController) — приём HTTP-запросов, валидация входных данных, подготовка модели для шаблонов Thymeleaf. Не содержит бизнес-логики;
- Service Layer (@Service) — инкапсуляция бизнес-правил, оркестрация портов, координация сложных операций;
- Port Layer (интерфейсы) — контракты доступа к данным;
- Adapter Layer (@Component) — реализации портов, работающие с конкретными механизмами хранения;
- Repository/Storage Layer — Spring Data JPA-репозитории (для DB) и файловое хранилище (для JSON);

Общая архитектурная схема представлена на рисунке 1.

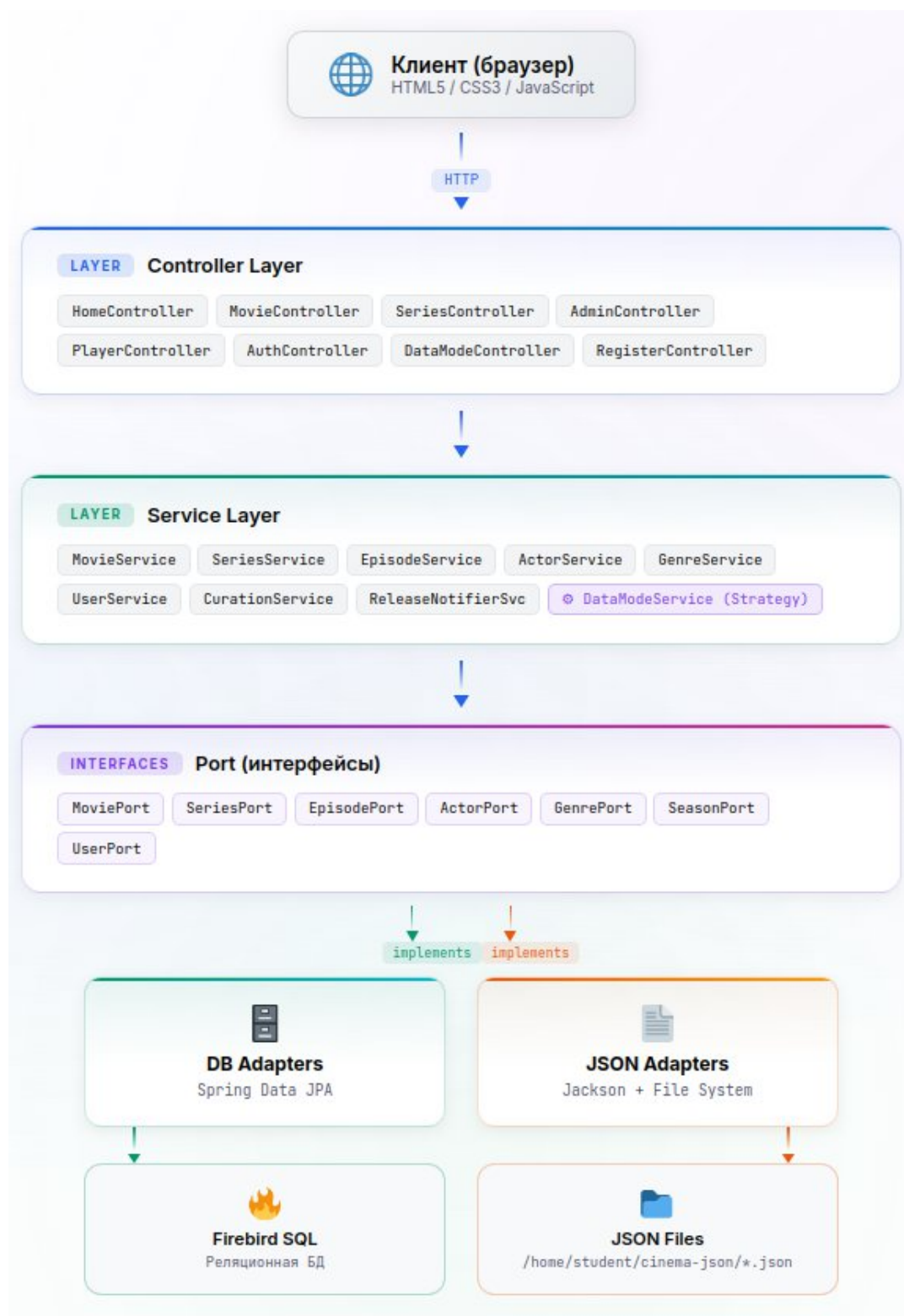


Рисунок 1 – Общая архитектурная схема CinemaHome (Port–Adapter)

Жизненный цикл данных в CinemaHome описывается следующими четырьмя этапами:

- Этап проектирования и создания — администратор или модератор через административную панель /admin/** добавляет фильмы, сериалы, жанры, актёров и режиссёров. Данные проходят валидацию на уровне контроллера и сохраняются в выбранное хранилище через соответствующий адаптер;
- Этап каталогизации — система индексирует добавленный контент, формирует связи между сущностями (фильм ↔ жанры, фильм ↔ актёры, сериал ↔ сезоны ↔ эпизоды) и сохраняет метаданные;
- Этап потребления — обычный пользователь (user) просматривает каталог через страницы /movies, /series, /genres, /actors и воспроизводит видео через HTML5-плеер (/player/movie/{id}, /player/series/{episodeId});
- Этап рекомендаций — CurationService на основе каталога и настройки пользователя (через MoodAnalyzer) формирует персональные рекомендации на главной странице /.

1.3 Функциональные требования к системе

Функциональные требования описывают конкретные возможности системы, которые должны быть реализованы для удовлетворения потребностей пользователей. В проекте CinemaHome выделены шесть подсистем, каждая из которых решает свой круг задач.

1.3.1 Подсистема хранения данных

Подсистема хранения данных является центральной для проекта, поскольку именно она обеспечивает ключевую особенность приложения — динамическую смену хранилища.

Основными требованиями являются:

- Поддержка двух независимых режимов хранения: DB (Firebird через Spring Data JPA / Hibernate) и JSON (файлы в директории /home/student/cinema-json/);

— Переключение режимов через отдельную страницу /mode с сохранением выбора в HttpSession (каждый пользователь может работать в своём режиме независимо от других);

— Идентичный контракт доступа к данным в обоих режимах (обеспечивается через паттерн Port–Adapter);

— Автоматическое создание структуры БД при первом запуске (через spring.jpa.hibernate.ddl-auto=update);

— Автоматическое создание директории /home/student/cinema-json/ при первом переключении в режим JSON.

1.3.2 Подсистема бизнес-логики (Backend)

Подсистема бизнес-логики инкапсулирует все правила обработки данных и оркестрирует взаимодействие между компонентами.

Основными требованиями являются:

— Управление тремя ролями пользователей: ADMIN, moderator, user с различным уровнем привилегий;

— CRUD-операции для фильмов и сериалов с каскадным созданием связанных сущностей (сезонов и эпизодов для сериалов);

— Автоматическое создание служебных пользователей admin/admin (роль ADMIN) и moderator/moderator (роль moderator) при старте приложения через AdminInitializer;

— Формирование персональных рекомендаций на основе случайного выбора из каталога с присвоением тега настроения;

— Система уведомлений о новых релизах через паттерн Observer с тремя каналами (Email, InApp, Mood).

1.3.3 Подсистема представления (Frontend)

Подсистема представления отвечает за визуализацию данных и взаимодействие с пользователем.

Основными требованиями являются:

- Серверный рендеринг HTML-страниц через шаблонизатор Thymeleaf с поддержкой Spring Expression Language (Spring EL);
- Минималистичная CSS-вёрстка с использованием шрифта Inter и приглушённой цветовой палитры;
- Встроенный HTML5-видеоплеер с раздачей медиафайлов через WebConfig (/media/**);
- Навигация по эпизодам сериала: кнопки «Предыдущая серия» и «Следующая серия», а также визуальная панель со всеми эпизодами сезона (CSS-классы .episode-square, .current-ep);
- Адаптивная вёрстка, корректно отображающаяся на десктопных и мобильных устройствах.

1.3.4 Регистрация и аутентификация

Основными требованиями являются:

- Регистрация новых пользователей через форму /register с обязательными полями «имя пользователя» и «пароль»;
- Вход в систему через форму /login с использованием стандартного механизма Spring Security;
- Хеширование паролей через BCryptPasswordEncoder перед сохранением в БД или JSON;
- Отображение сообщения «Неверный логин или пароль» при неудачной попытке входа (через редирект на /login?error=true с Thymeleaf-условием th:if="{param.error}");
- Интеграция с UserDetailsService через класс CustomUserDetailsService.

1.3.5 Ролевая модель

Роли и их привилегии представлены в таблице 3:

Таблица 3 — Роли и их привилегии

Роль	Привилегии
ADMIN	Полный доступ ко всем разделам, включая управление пользователями. Автоматически создаётся при старте приложения.

moderator	Доступ ко всем пяти разделам административной панели: управление фильмами (/admin/movies), сериалами (/admin/series), жанрами (/admin/genres), актёрами (/admin/actors), режиссёрами (/admin/directors). Автоматически создаётся при старте.
user	Просмотр каталога, детальное описание фильмов/сериалов, воспроизведение видео. Назначается по умолчанию при регистрации.

Обработка ошибок доступа:

- При попытке доступа к защищённому ресурсу без соответствующей роли пользователь перенаправляется на отдельную страницу /access-denied (HTTP 403) через AccessDeniedController;
- Конфигурация Spring Security включает exceptionHandling(ex -> ex.accessDeniedPage("/access-denied")).

1.3.6 Обработка ошибок и исключительных ситуаций

- Страница 404 (error.html) для несуществующих маршрутов с дружелюбным сообщением «Сцена не найдена»;
 - Страница 403 (access-denied.html) для случаев отсутствия прав доступа;
 - Корректная обработка некорректных ID в URL (например, /movies/abc)
- возврат на страницу 404;
- Обработка пустых каталогов — отображение блока «Фильмов пока нет» с приглашением зайти в панель админа.

1.4 Управление медиаконтентом

1.4.1 Поддерживаемые типы контента

В проекте CinemaHome реализована поддержка восьми типов сущностей, образующих иерархическую структуру медиакаталога.

- Фильм (Movie) — базовая сущность для одиночного видеоконтента.
 - Атрибуты: ID (Long, автоинкремент), Title (VARCHAR(255), обязательное), Description (BLOB, текстовое описание неограниченной длины), FilePath (VARCHAR(500), обязательное, путь к видеофайлу), CoverPath (VARCHAR(500), опциональное, путь к постеру), ReleaseYear (INTEGER), Status

(VARCHAR(20) с CHECK-ограничением: released/upcoming), Duration (INTEGER, минуты), IsWatched (BOOLEAN, по умолчанию FALSE);

- Связи: ManyToOne с Director (режиссёр), ManyToMany с Genre (жанры), ManyToMany с Actor (актёры);

- Бизнес-правило: фильм должен иметь как минимум название и путь к файлу.

- Сериал (Series) — сущность для многосерийного контента.

- Атрибуты: ID, Title, Description (BLOB), FilePath, CoverPath, Status (CHECK: released/ongoing), SeasonsCount, EpisodesCount, IsWatched;

- Связи: ManyToOne с Director, OneToMany с Season (каскадное удаление);

- Бизнес-правило: при создании сериала автоматически создаётся как минимум один сезон и один эпизод.

- Сезон (Season) — логическая группировка эпизодов в рамках сериала.

- Атрибуты: ID, SeasonNumber (обязательное), EpisodesCount, Status, FilePath, Duration, ReleaseYear, Title, IsWatched;

- Связи: ManyToOne с Series (ON DELETE CASCADE), ManyToOne с Director, OneToMany с Episode (каскадное удаление).

- Эпизод (Episode) — минимальная единица видеоконтента в сериале.

- Атрибуты: ID, EpisodeNumber (обязательное), Title, Duration, FilePath, IsWatched;

- Связи: ManyToOne с Season (ON DELETE CASCADE);

- Бизнес-правило: путь к файлу формируется автоматически по шаблону {folder}/{seasonNumber} сезон/{episodeNumber} серия.mp4.

- Актёр (Actor) — справочная сущность.

- Атрибуты: ID, FirstName, LastName, Bio (обязательное);

- Связи: ManyToMany с Movie через связующую таблицу MovieActors (с дополнительным атрибутом CharacterName).

- Жанр (Genre) — справочная сущность.

- Атрибуты: ID, Name (UNIQUE, обязательное), Description (BLOB);

- Связи: ManyToMany с Movie через связующую таблицу MovieGenres.
- Режиссёр (Director) — справочная сущность.
 - Атрибуты: ID, Name (UNIQUE, обязательное);
 - Связи: используется в полях DirectorID таблиц Movies и Seasons.
- Пользователь (User) — сущность системы безопасности.
 - Атрибуты: ID, Username (UNIQUE, обязательное), Password (хешированный), CreatedAt (LocalDateTime), RoleName (CHECK: ADMIN/moderator/user);
 - Бизнес-правило: при сохранении нового пользователя через @PrePersist автоматически устанавливается createdAt = LocalDateTime.now() и roleName = "user", если роль не указана явно.

1.4.2 Поддерживаемые типы контента

Административная панель включает пять разделов, которые представлены в таблице 4, доступных только пользователям с ролями MODERATOR и ADMIN (через аннотацию @PreAuthorize("hasRole('MODERATOR') or hasRole('ADMIN')")):

Таблица 4 — Панели разделов

URL	Назначение	Шаблон
/admin/movies	Управление фильмами	admin/form.html
/admin/series	Управление сериалами	admin/series-form.html
/admin/genres	Управление жанрами	admin/genre-form.html
/admin/actors	Управление актёрами	admin/actor-form.html
/admin/directors	Управление режиссёрами	admin/director-form.html

На главной странице / реализован блок быстрого перехода к админ-формам с выпадающим списком из 5 пунктов (Фильм, Сериал, Жанр, Актёр, Режиссёр) и JavaScript-обработчиком goToAddPage(), использующим конструкцию switch для навигации.

1.4.3 Просмотр и фильтрация

Главная страница /:

- Отображает «фильм дня» — случайно выбранный из каталога фильм с тегом настроения от MoodAnalyzer;
- Если каталог пуст, отображается блок-приглашение «Фильмов пока нет»;
- Содержит панель управления контентом (см. 1.d.2).

Списковые страницы:

- /movies — сетка карточек фильмов с постерами и тегами настроения;
- /series — список сериалов с описаниями и кнопкой «Смотреть»;
- /genres — список жанров;
- /actors — список актёров с указанием имени и фамилии.

Детальные страницы:

- /movies/{id} — детальная страница фильма с описанием, жанрами, актёрами и кнопкой «Смотреть»;
- /series/{id}/episodes — список эпизодов первого сезона выбранного сериала;
- /player/movie/{id} — HTML5-плеер для фильмов;
- /player/series/{episodeId} — HTML5-плеер для эпизодов с навигацией prev/next и панелью сезона.

1.4.4 Просмотр и фильтрация

Воспроизведение видео реализовано через стандартный HTML5-тег <video> с атрибутом controls:

```
<video width="800" controls>
  <source th:src="@{'/media/' + ${movie.filePath}}"
type="video/mp4">
  Ваш браузер не поддерживает воспроизведение видео.
</video>
```

Раздача медиафайлов обеспечивается конфигурацией WebConfig:

```
@Configuration
```



```

public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addResourceHandlers (ResourceHandlerRegistry
registry) {
        String videoLocation = "file:/home/student/videos/";
        registry.addResourceHandler ("/media/**")
            .addResourceLocations (videoLocation);
    }
}

```

Это позволяет обращаться к видеофайлам по URL /media/{filePath}, где filePath — относительный путь внутри директории /home/student/videos/.

1.4.5 Просмотр и фильтрация

Для сериалов реализована полноценная навигация:

- Кнопка «Предыдущая серия» — доступна, если текущий эпизод не является первым (реализована через метод findPrevious() в EpisodePort);
- Кнопка «Следующая серия» — доступна, если текущий эпизод не является последним (метод findNext());
- Панель навигации по сезону — визуальный ряд квадратных кнопок с номерами эпизодов (CSS-класс .episode-square), где текущий эпизод выделяется акцентным цветом (CSS-класс .current-ep).

Данная логика поддерживается обоими адаптерами:

- DB-адаптер: использует Spring Data-методы findBySeason_IdAndEpisodeNumber(seasonId, episodeNumber ± 1);
- JSON-адаптер: выполняет фильтрацию по seasonId и episodeNumber через Stream API.

2 Проектирование программы

2.1 Проектирование архитектуры и структуры данных системы

На начальном этапе разработки была проведена детальная формализация задач проекта. Программа должна решать комплекс задач: от динамического построения структуры медиакаталога администратором до обеспечения защищённого доступа пользователей и последующего воспроизведения видеоконтента. Ключевым требованием стала необходимость создания гибкой системы хранения данных, способной переключаться между реляционной СУБД и файловым хранилищем без перезапуска приложения и без модификации бизнес-логики.

Дополнительно требовалось обеспечить расширяемость архитектуры, чтобы в будущем можно было интегрировать новые типы хранилищ (например, MongoDB, PostgreSQL или облачные решения S3) без изменения существующего ядра системы. Этот аспект особенно важен в контексте учебного проекта, где демонстрируемая архитектура должна служить наглядным примером современных промышленных практик.

Сравнительный анализ архитектурных подходов

Для решения поставленных задач был проведён сравнительный анализ нескольких архитектурных подходов, представленных в таблице 5.

Таблица 5 — Сравнительный анализ архитектурных подходов

Архитектурный подход	Описание	Преимущества	Недостатки	Применимость к проекту
MVC (Model-View-Controller)	Классический паттерн разделения ответственности на модель данных, представление и контроллер [4]	Простота понимания, широкая распространённость	Жёсткая связь между бизнес-логикой и механизмами хранения	Частично (используется в Controller Layer)
Layered Architecture	Разделение на Presentation,	Чёткая иерархия, простота тестиро-	Не решает проблему зависимости	Используется как основа

(слоёная архитектура)	Business, Persistence, Database слои	вания	бизнес-логики от конкретных реализаций	
Clean Architecture (чистая архитектура) Р. Мартина	Концентрическая модель, где бизнес-правила находятся в центре и не зависят от внешних слоёв [5]	Строгое соблюдение DIP, высокая тестируемость	Избыточная сложность для учебного проекта	Частично заимствованы идеи
Hexagonal Architecture (Ports & Adapters) А. Коуборна	Бизнес-логика зависит только от абстрактных портов, конкретные реализации подключаются через адаптеры [2]	Идеально для задачи динамической смены хранилища, высокая расширяемость	Требует дисциплины при проектировании	Выбрана как основная
Onion Architecture	Развитие Clean Architecture с акцентом на инверсию зависимостей	Строгая изоляция доменной модели	Сложность для небольших проектов	Не выбрана из-за избыточности

На основе проведённого анализа была выбрана гексагональная архитектура (Ports & Adapters), предложенная Алистером Коуборном в 2005 году [2]. Данный выбор обоснован следующими факторами:

~ Прямое решение ключевой задачи проекта — паттерн Port–Adapter естественным образом обеспечивает возможность переключения между хранилищами без изменения бизнес-логики;

~ Соответствие принципам SOLID, в частности Dependency Inversion Principle (DIP) и Open/Closed Principle (OCP);

~ Высокая тестируемость — бизнес-логика может быть протестирована изолированно с использованием mock-адаптеров;

~ Низкий порог вхождения — архитектура проста для понимания и реализации в рамках учебного проекта.

Проектирование архитектуры строго следует принципам SOLID, сформулированным Робертом Мартином [5]:

Single Responsibility Principle (SRP) — каждый класс имеет единственную ответственность:

~ MovieService — оркестрирует бизнес-логику фильмов;

- ~ MoviePort — определяет контракт доступа к данным фильмов;
- ~ DbMovieAdapter — реализует доступ через Spring Data JPA;
- ~ JsonMovieAdapter — реализует доступ через файловое хранилище.

Open/Closed Principle (OCP) — система открыта для расширения, но закрыта для модификации. Добавление нового хранилища (например, MongoDB) требует лишь создания класса MongoMovieAdapter, реализующего MoviePort, без изменения сервисного слоя.

Liskov Substitution Principle (LSP) — любой адаптер может быть подставлен вместо другого без нарушения корректности программы. Все реализации MoviePort имеют идентичный контракт.

Interface Segregation Principle (ISP) — интерфейсы портов минималистичны и содержат только необходимые методы. MoviePort определяет всего три операции: findAll(), findById(), save().

Dependency Inversion Principle (DIP) — бизнес-логика зависит от абстракций (портов), а не от конкретных реализаций (адаптеров). Конкретные адаптеры внедряются через @Autowired + @Qualifier.

Архитектурная модель системы может быть представлена в виде UML-диаграммы классов, иллюстрирующей взаимоотношения между основными компонентами (рисунок 2).

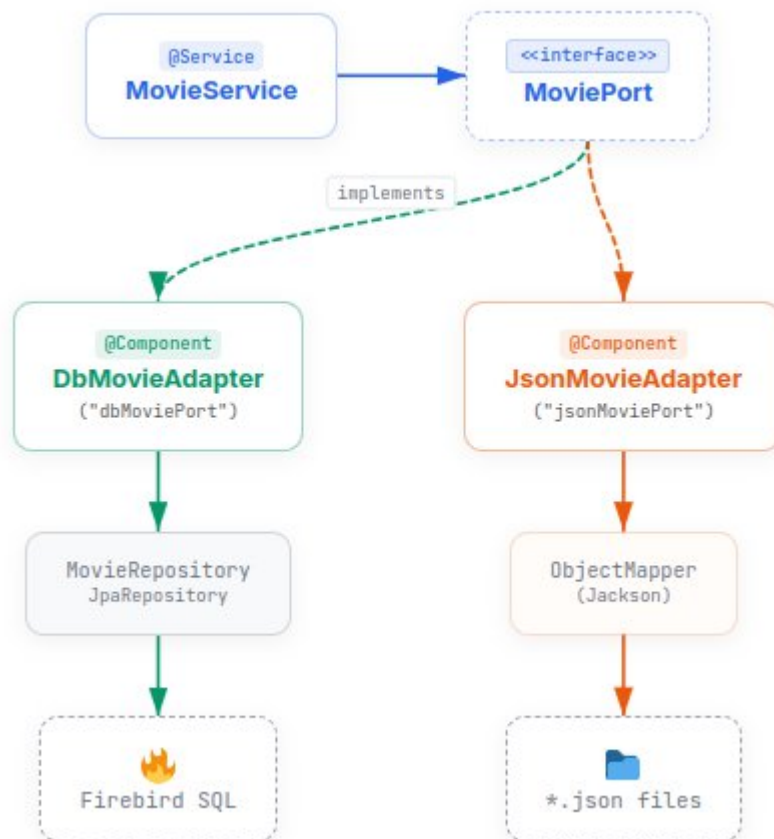


Рисунок 2 — Схема взаимодействия программных Java-интерфейсов и слоёв

Аналогичная структура применяется ко всем 8 сущностям системы: Movie, Series, Season, Episode, Actor, Genre, Director, User. Каждая сущность имеет собственный порт и две параллельные реализации адаптеров.

Для иллюстрации взаимодействия компонентов рассмотрим сценарий добавления нового фильма через административную панель. Диаграмма последовательностей представлена на рисунке 3.

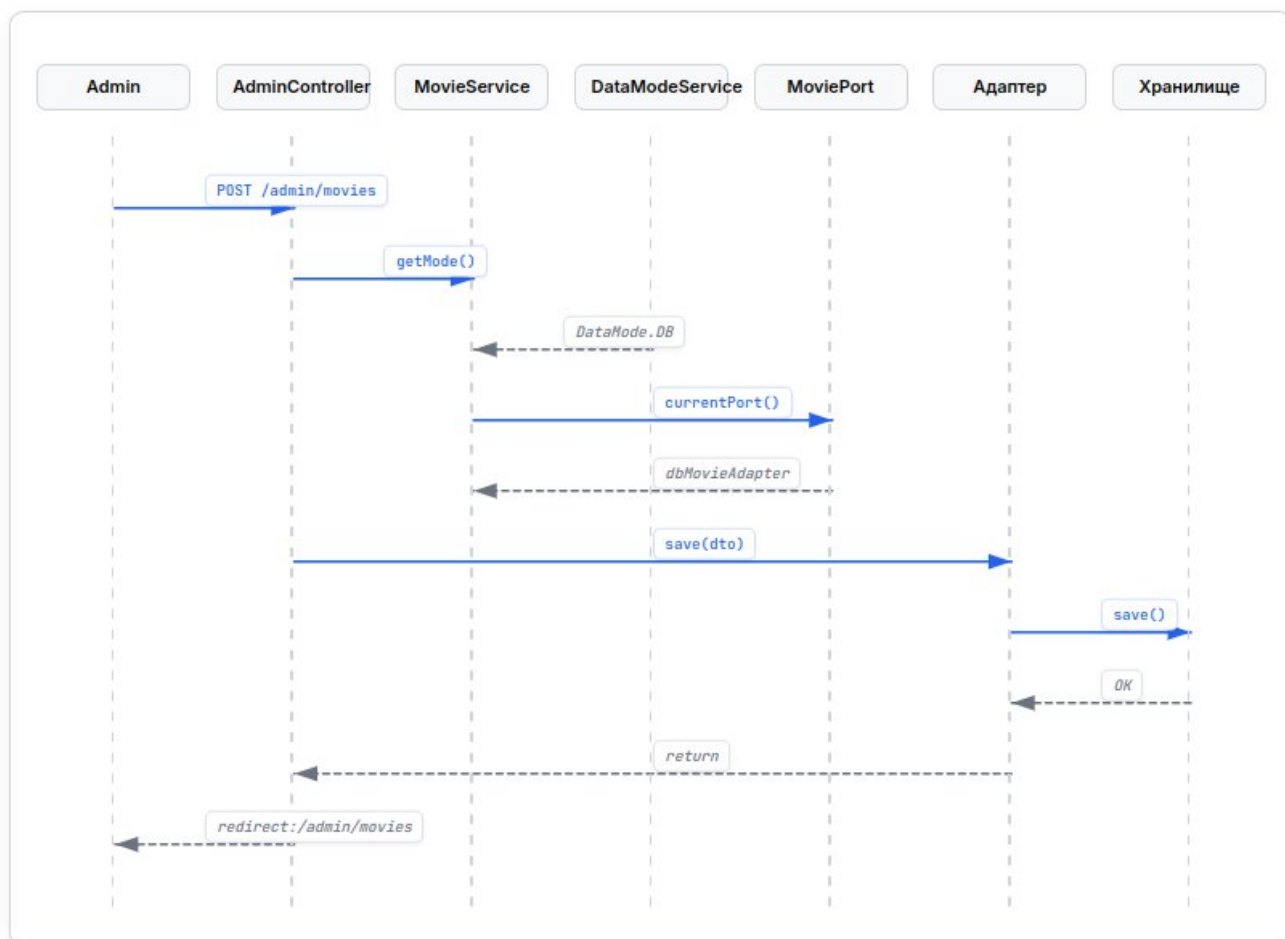


Рисунок 3 — Диаграмма последовательностей: добавление фильма

Из диаграммы видно, что бизнес-логика (MovieService) не имеет прямого доступа к хранилищу — она взаимодействует исключительно через абстракцию MoviePort. Конкретный адаптер выбирается динамически на основе значения DataMode в HttpSession.

Для сущностей Movie и Series были спроектированы диаграммы состояний (state machine), описывающие жизненный цикл контента (рисунок 4).

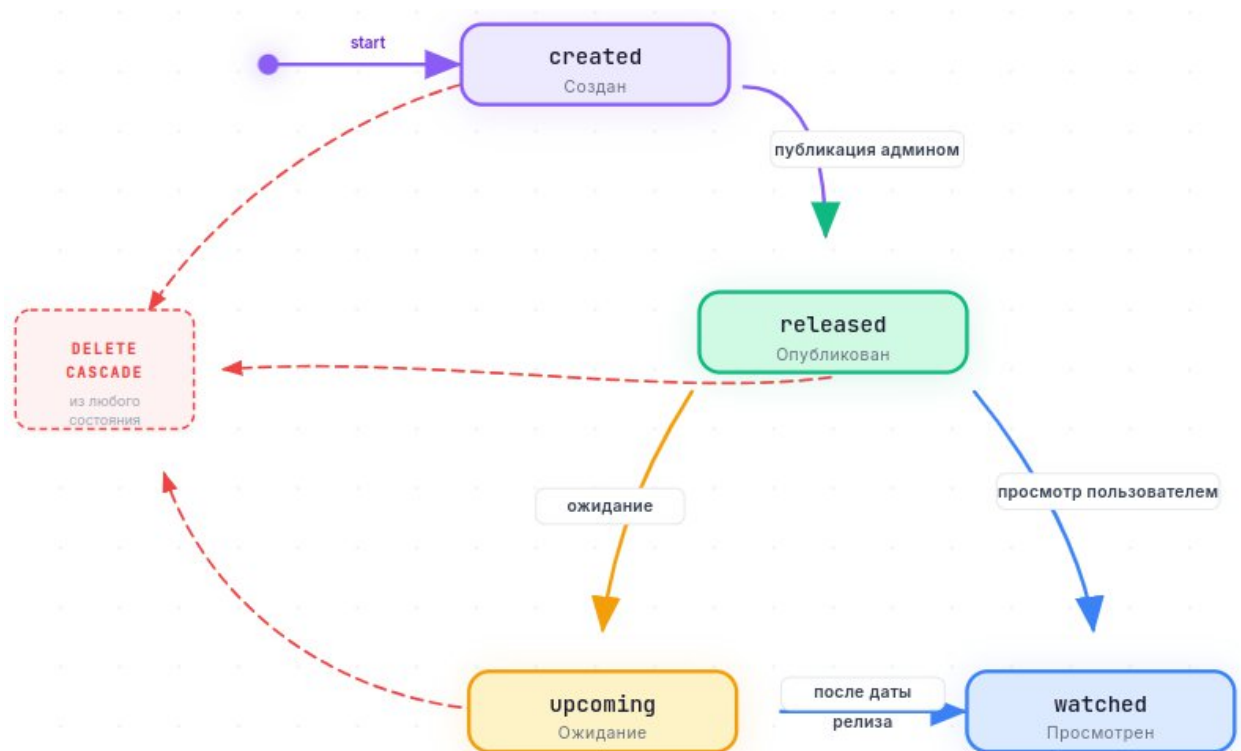


Рисунок 4 — Диаграмма состояний: жизненный цикл фильма

Для сериалов используется статус `ongoing` вместо `upcoming`, что отражает специфику многосерийного контента. Состояния `released` и `ongoing` проверяются на уровне БД через CHECK-ограничения:

```

Status VARCHAR(20) CHECK (Status IN ('released', 'upcoming'))
-- для Movies
Status VARCHAR(20) CHECK (Status IN ('released', 'ongoing'))
-- для Series

```

Общая диаграмма компонентов системы представлена на рисунке 5.

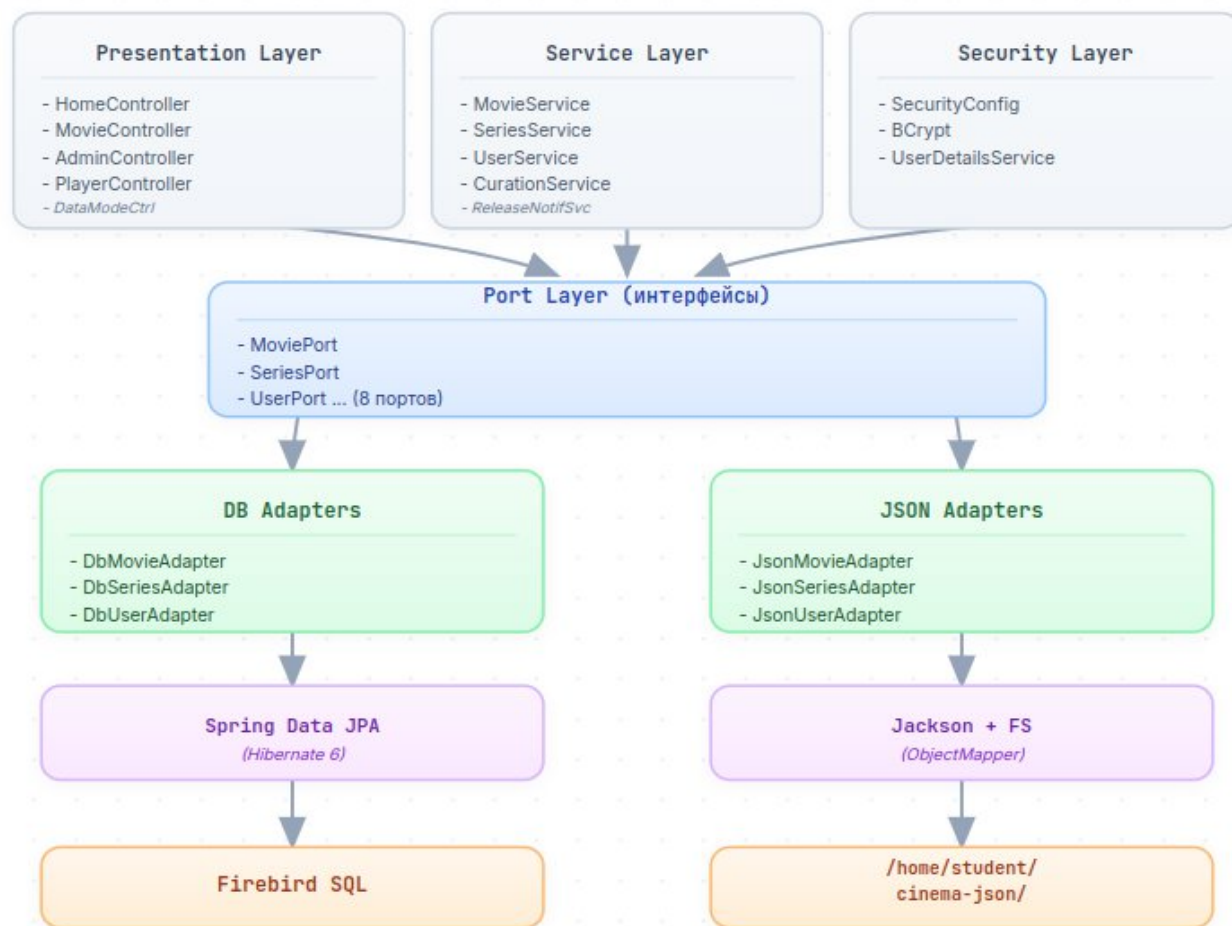


Рисунок 5 — Диаграмма компонентов системы CinemaHome

В проекте применяются несколько классических паттернов проектирования, каждый из которых решает свою специфическую задачу:

1. Port-Adapter (Hexagonal Architecture) — центральный паттерн проекта. Обеспечивает инверсию зависимостей и возможность динамической смены хранилища. Ключевые интерфейсы: MoviePort, SeriesPort, EpisodePort, SeasonPort, ActorPort, GenrePort, DirectorPort, UserPort.
2. Strategy (Стратегия) — поведенческий паттерн, позволяющий выбирать алгоритм в runtime. Реализован через класс DataModeService, который хранит выбранный режим (DB или JSON) в HttpSession и предоставляет сервисам метод currentPort(), возвращающий нужный адаптер. Это обеспечивает соблюдение принципа OCP.

3. Observer (Наблюдатель) — поведенческий паттерн, реализующий механизм подписки на события. Используется для системы уведомлений о релизах.

Интерфейс ReleaseNotifier имеет три реализации:

- ~ EmailNotifier (основной, помечен @Primary) — имитация отправки email;
- ~ InAppNotifier — уведомление внутри приложения;
- ~ MoodNotifier — персонализированное уведомление с учётом настроек пользователя через MoodAnalyzer.

4. Dependency Injection / IoC — все компоненты получают зависимости через контейнер Spring, что обеспечивает слабую связанность и высокую тестируемость.

5. Template Method (через Spring Data) — общий шаблон CRUD-операций реализуется базовым интерфейсом JpaRepository<T, ID>, а конкретные репозитории добавляют лишь специфические методы запросов (например, findBySeason_IdAndEpisodeNumber в EpisodeRepository).

Presentation Layer (Controller Layer):

- ~ Обработывает HTTP-запросы и готовит модель для шаблонов Thymeleaf;
- ~ Использует аннотации @Controller, @GetMapping, @PostMapping, @PathVariable, @RequestParam;
- ~ Применяет @PreAuthorize для декларативной проверки ролей;
- ~ Не содержит бизнес-логики, только валидация входных данных и маршрутизация.

Service Layer:

- ~ Инкапсулирует всю бизнес-логику;
- ~ Орkestрирует взаимодействие между портами;
- ~ Содержит вспомогательные компоненты (MoodAnalyzer, CurationService);
- ~ Каждый сервис имеет метод currentPort(), возвращающий нужный адаптер на основе DataMode.

Port Layer:

- ~ Java-интерфейсы, определяющие минимально необходимый контракт доступа к данным;
- ~ Не содержат никакой реализации — только сигнатуры методов;
- ~ Обеспечивают инверсию зависимостей (DIP).

Adapter Layer:

- ~ Конкретные реализации портов;
- ~ Db*Adapter — работа через Spring Data JPA с Firebird;
- ~ Json*Adapter — работа через Jackson Databind с файлами *.json в директории /home/student/cinema-json/;
- ~ Оба типа адаптеров имеют идентичный контракт, что позволяет взаимозаменять их без изменения вызывающего кода.

Repository / Storage Layer:

- ~ Spring Data JPA-репозитории для БД-режима;
- ~ Файловое хранилище для JSON-режима;
- ~ Обеспечивают низкоуровневый доступ к данным.

Security Layer:

- ~ Spring Security для аутентификации и авторизации;
- ~ BCryptPasswordEncoder для хеширования паролей;
- ~ CustomUserDetailsService для интеграции с репозиторием пользователей;
- ~ Декларативная проверка ролей через @PreAuthorize.

Config Layer:

- ~ DbConfig — конфигурация DataSource для Firebird с аннотацией @ConditionalOnMissingBean для корректной работы тестов;
- ~ WebConfig — раздача медиафайлов через /media/**;
- ~ SecurityConfig — настройка цепочки фильтров безопасности;
- ~ OpenApiConfig — генерация OpenAPI-спецификации для Swagger UI;
- ~ AdminInitializer — автоматическое создание служебных пользователей при старте;

~ DataModeService — управление режимом хранения.

Таким образом, в основе архитектуры проектируемой программы лежит система интерфейсов и слоёв, через которые взаимодействуют все компоненты. Благодаря такой модульной архитектуре каждый компонент выполняет строго ограниченную задачу, что позволяет системе быть гибкой, отказоустойчивой и готовой к дальнейшему масштабированию.

2.2 Проектирование информационной модели и базы данных

В качестве модели хранения была выбрана реляционная модель [3], обеспечивающая целостность, непротиворечивость и минимальную избыточность.

2.2.1 Сущность «Фильм» (Movies)

Центральный объект системы. Атрибуты: ID (VARCHAR(50), PK), Title, Description (BLOB), FilePath (NOT NULL), CoverPath, ReleaseYear, Status (CHECK: released/upcoming), Duration, IsWatched, DirectorID (FK → Directors).

2.2.2 Сущность «Сериал» (Series)

Содержит метаданные о многосерийном контенте. Атрибуты: ID, Title, Description (BLOB), FilePath, CoverPath, Status (CHECK: released/ongoing), SeasonsCount, EpisodesCount, IsWatched.

2.2.3 Сущности «Сезон» (Seasons) и «Эпизод» (Episodes)

Реализуют иерархию «Сериал → Сезон → Эпизод» со связями ON DELETE CASCADE.

2.2.4 Справочники: Genres, Actors, Directors

~ Genres — ID (IDENTITY), Name (UNIQUE);

~ Actors — ID, first_name, last_name, bio;

~ Directors — ID, Name (UNIQUE).

2.2.5 Связующие таблицы

~ MovieGenres (MovieID, GenreID) — многие-ко-многим;

~ MovieActors (MovieID, ActorID, CharacterName) — многие-ко-многим с дополнительным атрибутом.

2.2.6 Сущность «Пользователь» (Users)

~ ID, Username (UNIQUE), RoleName (CHECK: ADMIN/moderator/user), PasswordHash, Email.

2.2.7 История и избранное

~ UserMovieHistory, UserSeriesHistory, UserEpisodeHistory — история просмотров с рейтингом;

~ UserFavorites, UserFavoriteSeries — избранное.

2.2.8 Целостность данных и индексы

~ ON DELETE CASCADE для всех дочерних связей;

~ Индексы по полям Title, ReleaseYear, Status, IsWatched, внешним ключам и полям поиска.

Финальная ER-диаграмма представлена на рисунке 6.

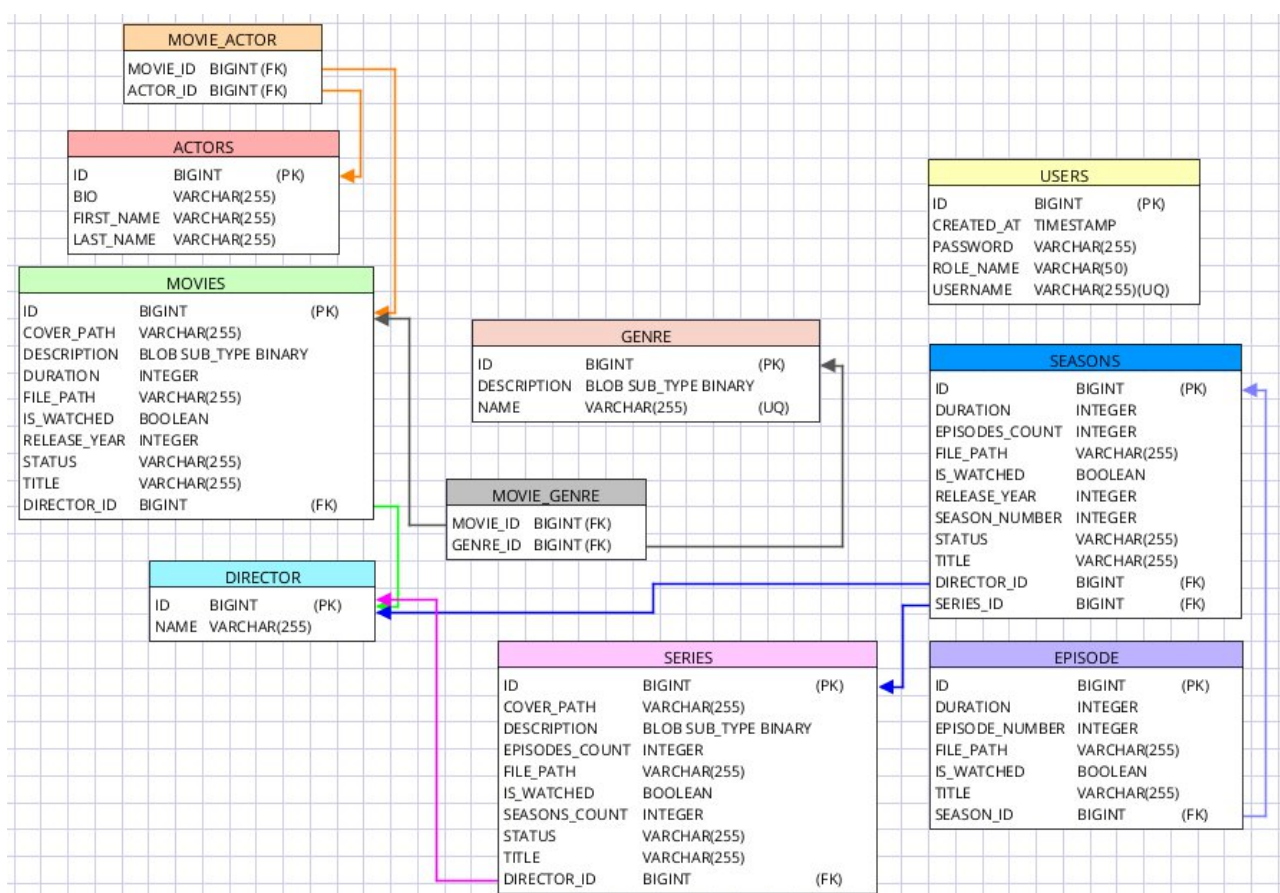


Рисунок 6 - ER-диаграмма

3 Разработка приложения

3.1 Выбор и обоснование технологического стека

3.1.1 Обоснование выбора языка программирования

В качестве основного языка программирования была выбрана Java 21 — актуальный LTS-релиз с долгосрочной поддержкой до сентября 2031 года. Данный выбор обусловлен несколькими факторами:

- Долгосрочная поддержка (LTS): версия 21 будет получать обновления безопасности и исправления ошибок в течение 8 лет, что критично для промышленного применения;
- Современные языковые конструкции: доступ к record patterns, pattern matching for switch, sealed classes, virtual threads (Project Loom) — всё это повышает выразительность и производительность кода;
- Строгая статическая типизация: позволяет выявлять ошибки на этапе компиляции, а не во время выполнения;
- Кроссплатформенность: принцип WORA (Write Once, Run Anywhere) обеспечивает работу приложения на любой ОС с установленной JVM;
- Зрелая экосистема: тысячи готовых библиотек, фреймворков и инструментов разработки.

3.1.2 Фреймворк Spring Boot 3.4.0

Приложение построено на базе Spring Boot 3.4.0 — актуальной версии фреймворка с полной поддержкой спецификации Jakarta EE 10. Ключевые преимущества:

- Автоконфигурация: Spring Boot автоматически настраивает компоненты на основе обнаруженных в classpath зависимостей;
- Стартеры: готовые модули spring-boot-starter-web, spring-boot-starter-data-jpa, spring-boot-starter-security, spring-boot-starter-thymeleaf значительно ускоряют разработку;

- Embedded-сервер: Tomcat 10.1 встроен в приложение, что позволяет запускать его одной командой без предварительной установки веб-сервера;
- Actuator: встроенные эндпоинты для мониторинга состояния приложения (/actuator/health, /actuator/info).

3.1.3. Сборка и управление зависимостями

Управление зависимостями и сборка делегированы Gradle 8.5 с официальным Spring Plugin версии 3.4.0. Ключевые преимущества Gradle перед Maven:

- Гибкий DSL: на базе Groovy/Kotlin, позволяющий писать декларативные скрипты сборки;
- Инкрементальная сборка: Gradle отслеживает изменения файлов и пересобирает только изменённые модули;
- Gradle Wrapper: включён в репозиторий (gradlew, gradlew.bat), что обеспечивает воспроизводимость сборки на любых машинах без предварительной установки Gradle.

3.1.4. Работа с базой данных

Для работы с реляционной СУБД используется Firebird 3.0+ с нативным JDBC-драйвером Jaybird 5.0.11. Данная СУБД выбрана по следующим причинам:

- Легковесность: установочный пакет ~15 МБ, не требует выделенного серверного процесса в embedded-режиме;
- Кроссплатформенность: работает на Linux, Windows, macOS;
- Поддержка BLOB-полей: критично для хранения объёмных текстовых описаний фильмов и сериалов;
- Простота администрирования: минимальные требования к квалификации DBA.

Взаимодействие с БД организовано через Hibernate 6.6.2.Final в связке с Community Dialects, что обеспечивает корректную генерацию SQL-запросов под специфичный диалект Firebird. Конфигурация подключения задаётся в файле application.properties:

3.1.5. Сериализация JSON

					МИВУ.09.03.02 009 ПЗ	Лист
						36
Изм.	Лист	№ докум.	Подпись	Дата		

Сериализация и десериализация данных в формате JSON осуществляется с помощью Jackson Databind 2.17.0 — де-факто стандарта в экосистеме Spring. Ключевые возможности:

- Глубокая интеграция со Spring Boot — автоматическая настройка ObjectMapper;
- Поддержка полиморфных типов через аннотации @JsonTypeInfo, @JsonSubTypes;
- Кастомные сериализаторы/десериализаторы для сложных типов данных;
- Pretty-printer (writerWithDefaultPrettyPrinter()) для удобочитаемого вывода JSON-файлов.

3.1.6. Шаблонизатор Thymeleaf

Для рендеринга серверных страниц применяется Thymeleaf, позволяющий использовать «натуральные» HTML-шаблоны с поддержкой Spring Expression Language (Spring EL). Преимущества:

- Валидный HTML: шаблоны остаются корректными HTML-файлами даже без серверной обработки;
- Тесная интеграция со Spring MVC и Spring Security через пространство имён sec::;
- Fragment Layouts: возможность переиспользовать общие части страницы (шапку, подвал, навигацию).

3.1.7. Безопасность

Безопасность приложения реализована на базе Spring Security совместно с BCrypt. Настроена:

- Ролевая модель авторизации (ADMIN, moderator, user);
- Защита от CSRF-атак (временно отключена для упрощения POST-запросов на /mode);
- Управление пользовательскими сессиями через HttpSession;
- Хеширование паролей с адаптивной функцией BCrypt.

3.1.8. Сокращение boilerplate-кода

					МИВУ.09.03.02 009 ПЗ	Лист
						37
Изм.	Лист	№ докум.	Подпись	Дата		

Для сокращения шаблонного кода (геттеров, сеттеров, конструкторов, equals, hashCode, toString) активно используется Lombok 1.18.36. Аннотации @Data, @NoArgsConstructor, @AllArgsConstructor, @Getter, @Setter позволяют заменить десятки строк шаблонного кода одной аннотацией над классом.

3.1.9. API-документация

Документация REST API генерируется автоматически через springdoc-openapi 2.3.0. Интерактивный Swagger UI доступен по пути /swagger-ui.html, а OpenAPI-спецификация в формате JSON — по пути /api-docs.

3.1.10. Тестирование

Тестовое покрытие обеспечивается связкой:

- JUnit 5 — современный стандарт модульного тестирования в Java;
- Mockito 5+ — мокирование зависимостей;
- AssertJ — fluent-стиль утверждений (assertThat(...).isEqualTo(...));
- Spring Boot Test — интеграционные тесты с загрузкой полного контекста.

3.2 Реализация системы адаптеров хранилища (Port–Adapter)

Центральным элементом архитектуры CinemaHome является гексагональная архитектура (Ports & Adapters), обеспечивающая возможность динамической смены хранилища данных без изменения бизнес-логики. Для каждой из 8 сущностей системы (Movie, Series, Season, Episode, Actor, Genre, Director, User) реализована пара адаптеров — DB и JSON.

3.2.1. Интерфейсы-порты

Порты определяют минимально необходимый контракт доступа к данным. Все порты находятся в пакете org.example.cinemahome.port и являются обычными Java-интерфейсами.

```
package org.example.cinemahome.port;
```



```

import org.example.cinemahome.dto.MovieDto;
import java.util.List;

public interface MoviePort {
    List<MovieDto> findAll();
    MovieDto findById(Long id);
    void save(MovieDto movie);
}

package org.example.cinemahome.port;

import org.example.cinemahome.dto.EpisodeDto;
import java.util.List;

public interface EpisodePort {
    EpisodeDto findById(Long id);
    EpisodeDto findNext(EpisodeDto current);
    EpisodeDto findPrevious(EpisodeDto current);
    EpisodeDto findFirstOfSeries(Long seriesId);
    List<EpisodeDto> findBySeasonId(Long seasonId);
}

```

Особенностью порта EpisodePort является наличие методов findNext() и findPrevious(), которые используются для навигации между эпизодами сериала. Эти методы реализованы как в DB-, так и в JSON-адаптерах, что обеспечивает одинаковое поведение системы независимо от выбранного режима хранения.

3.2.2. DB-адаптеры (Spring Data JPA)

DB-адаптеры работают с СУБД Firebird через Spring Data JPA. Каждый адаптер помечен аннотацией @Component с явным указанием имени бина через параметр ("db*Port") — это необходимо для последующего внедрения через @Qualifier.

```

package org.example.cinemahome.adapter.db;

```

					МИВУ.09.03.02 009 ПЗ	Лист
						39
Изм.	Лист	№ докум.	Подпись	Дата		

```

import org.example.cinemahome.dto.MovieDto;
import org.example.cinemahome.pojo.Movie;
import org.example.cinemahome.port.MoviePort;
import org.example.cinemahome.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import java.util.List;
import java.util.stream.Collectors;

@Component("dbMoviePort")
public class DbMovieAdapter implements MoviePort {
    @Autowired private MovieRepository movieRepository;
    @Autowired private GenreRepository genreRepository;
    @Autowired private ActorRepository actorRepository;
    @Override
    public List<MovieDto> findAll() {
        return movieRepository.findAll().stream()
            .map(this::toDto)
            .collect(Collectors.toList());
    }
    @Override
    public MovieDto findById(Long id) {
        return movieRepository.findById(id)
            .map(this::toDto)
            .orElse(null);
    }
    @Override
    public void save(MovieDto dto) {
        Movie m = new Movie();
        if (dto.getId() != null) m.setId(dto.getId());
        m.setTitle(dto.getTitle());
        m.setDescription(dto.getDescription());
        m.setFilePath(dto.getFilePath());
    }
}

```

```

        if (dto.getGenreIds() != null)

            m.setGenres(genreRepository.findAllById(dto.getGenreIds()));

            if (dto.getActorIds() != null)

                m.setActors(actorRepository.findAllById(dto.getActorIds()));

            movieRepository.save(m);
        }
        private MovieDto toDto(Movie movie) {
            MovieDto dto = new MovieDto();
            dto.setId(movie.getId());
            dto.setTitle(movie.getTitle());
            dto.setDescription(movie.getDescription());
            dto.setFilePath(movie.getFilePath());
            dto.setGenreIds(movie.getGenres().stream()
                .map(g ->
g.getId()).collect(Collectors.toList()));
            dto.setActorIds(movie.getActors().stream()
                .map(a ->
a.getId()).collect(Collectors.toList()));
            return dto;
        }
    }

package org.example.cinemahome.adapter.db;

import org.example.cinemahome.dto EpisodeDto;
import org.example.cinemahome.pojo Episode;
import org.example.cinemahome.port EpisodePort;
import org.example.cinemahome.repository EpisodeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import java.util.List;

```

```

@Component("dbEpisodePort")
public class DbEpisodeAdapter implements EpisodePort {
    @Autowired private EpisodeRepository episodeRepository;
    private EpisodeDto toDto(Episode e) {
        return new EpisodeDto(
            e.getId(), e.getTitle(), e.getFilePath(),
            e.getEpisodeNumber(), e.getSeason().getId()
        );
    }
    @Override
    public EpisodeDto findNext(EpisodeDto current) {
        if (current == null || current.getSeasonId() == null
            || current.getEpisodeNumber() == null) return
null;

        return episodeRepository
            .findBySeason_IdAndEpisodeNumber(
                current.getSeasonId(),
                current.getEpisodeNumber() + 1
            )
            .map(this::toDto).orElse(null);
    }
    @Override
    public EpisodeDto findPrevious(EpisodeDto current) {
        if (current == null || current.getSeasonId() == null
            || current.getEpisodeNumber() == null
            || current.getEpisodeNumber() <= 1) return
null;

        return episodeRepository
            .findBySeason_IdAndEpisodeNumber(
                current.getSeasonId(),
                current.getEpisodeNumber() - 1
            )
            .map(this::toDto).orElse(null);
    }
}

```

```

        // ... остальные методы (findById, findFirstOfSeries,
        findByIdBySeasonId)
    }

```

Методы `findByIdSeason_IdAndEpisodeNumber` используют механизм `derived queries` Spring Data JPA — фреймворк автоматически строит SQL-запрос на основе имени метода, обходясь без явного написания `@Query`.

```

package org.example.cinemahome.adapter.db;

import org.example.cinemahome.dto.DirectorDto;
import org.example.cinemahome.pojo.Director;
import org.example.cinemahome.port.DirectorPort;
import org.example.cinemahome.repository.DirectorRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import java.util.List;
import java.util.stream.Collectors;

@Component("dbDirectorPort")
public class DbDirectorAdapter implements DirectorPort {
    @Autowired private DirectorRepository directorRepository;

    @Override
    public List<DirectorDto> findAll() {
        return directorRepository.findAll().stream()
            .map(d -> new DirectorDto(d.getId(), d.getName()))
            .collect(Collectors.toList());
    }

    @Override
    public void save(DirectorDto dto) {
        Director d = new Director();
        if (dto.getId() != null) d.setId(dto.getId());
        d.setName(dto.getName());
    }
}

```

```

        directorRepository.save(d);
    }
}

```

3.2.3. JSON-адаптеры (файловое хранилище)

JSON-адаптеры работают с файлами в директории /home/student/cinema-json/. Каждый адаптер:

- Помечен аннотацией `@Component("json*Port")`;
- Хранит путь к JSON-файлу в приватном поле;
- Использует `ObjectMapper` из Jackson для сериализации/десериализации;
- Автоматически создаёт родительские директории при первом запуске.

```

package org.example.cinemahome.adapter.json;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.example.cinemahome.dto.MovieDto;
import org.example.cinemahome.port.MoviePort;
import org.springframework.stereotype.Component;
import java.io.File;
import java.util.ArrayList;
import java.util.List;

@Component("jsonMoviePort")
public class JsonMovieAdapter implements MoviePort {
    // Намеренно без final — для подмены в тестах через
    ReflectionTestUtils
    private static String MOVIES_JSON_PATH =
        "/home/student/cinema-json/movies.json";
    private final ObjectMapper objectMapper = new
    ObjectMapper();

    private List<MovieDto> readAllInternal() {
        try {

```

```

        File f = new File(MOVIES_JSON_PATH);
        if (!f.exists()) return new ArrayList<>();
        return objectMapper.readValue(f,
            new TypeReference<List<MovieDto>>() {});
    } catch (Exception e) {
        e.printStackTrace();
        return new ArrayList<>();
    }
}

private void writeAllInternal(List<MovieDto> list) {
    try {
        File f = new File(MOVIES_JSON_PATH);
        File parent = f.getParentFile();
        if (parent != null && !parent.exists())
parent.mkdirs();

        objectMapper.writerWithDefaultPrettyPrinter().writeValue(f,
list);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

@Override
public List<MovieDto> findAll() {
    return readAllInternal();
}

@Override
public MovieDto findById(Long id) {
    return readAllInternal().stream()
        .filter(m -> id.equals(m.getId()))
        .findFirst()
        .orElse(null);
}

```

```

    }

    @Override
    public void save(MovieDto movie) {
        List<MovieDto> all = readAllInternal();
        if (movie.getId() == null) {
            long newId = all.stream()
                .mapToLong(m -> m.getId() == null ? 0L :
m.getId())
                .max().orElse(0L) + 1;
            movie.setId(newId);
        } else {
            all.removeIf(m ->
movie.getId().equals(m.getId()));
        }
        all.add(movie);
        writeAllInternal(all);
    }
}

```

Ключевые особенности JSON-адаптера:

- Автогенерация ID: при сохранении объекта без id адаптер вычисляет $\max(\text{существующих ID}) + 1$;
- Обновление записей: если объект с таким ID уже существует, он удаляется из списка перед добавлением новой версии (эмуляция UPDATE)¹;
- Pretty-printing: JSON-файлы записываются с форматированием, что облегчает ручное редактирование и отладку;
- Автоматическое создание директорий: при первом запуске, если директория `/home/student/cinema-json/` не существует, она создаётся автоматически.

```

@Override
public EpisodeDto findNext(EpisodeDto current) {
    if (current == null || current.getSeasonId() == null

```



```

        || current.getEpisodeNumber() == null) return
null;

        return readAllInternal().stream()
            .filter(e ->
current.getSeasonId().equals(e.getSeasonId())
            && (current.getEpisodeNumber() + 1) ==
e.getEpisodeNumber())
            .findFirst()
            .map(this::toDto)
            .orElse(null);
    }

    @Override
    public EpisodeDto findPrevious(EpisodeDto current) {
        if (current == null || current.getSeasonId() == null
            || current.getEpisodeNumber() == null
            || current.getEpisodeNumber() <= 1) return null;
        return readAllInternal().stream()
            .filter(e ->
current.getSeasonId().equals(e.getSeasonId())
            && (current.getEpisodeNumber() - 1) ==
e.getEpisodeNumber())
            .findFirst()
            .map(this::toDto)
            .orElse(null);
    }
}

```

В JSON-адаптере навигация реализована через Stream API с фильтрацией по `seasonId` и `episodeNumber`, что полностью соответствует семантике DB-адаптера.

3.2.4. Механизм выбора адаптера (@Qualifier + Strategy)

Для выбора нужного адаптера в runtime используется комбинация аннотаций `@Autowired` + `@Qualifier` и паттерна Strategy. В каждом сервисе присутствуют два поля-зависимости для каждого порта:

					МИВУ.09.03.02 009 ПЗ	Лист
						47
Изм.	Лист	№ докум.	Подпись	Дата		

```

@Service
public class MovieService {
    @Autowired @Qualifier("dbMoviePort")
    private MoviePort dbMovieAdapter;

    @Autowired @Qualifier("jsonMoviePort")
    private MoviePort jsonMovieAdapter;

    @Autowired
    private DataModeService dataModeService;

    private MoviePort currentPort() {
        DataMode mode = dataModeService.getMode();
        return (mode == DataMode.JSON) ? jsonMovieAdapter :
dbMovieAdapter;
    }

    public List<MovieDto> getAllMovies() {
        return currentPort().findAll();
    }

    public MovieDto getMovieById(Long id) {
        return currentPort().findById(id);
    }

    public void addMovie(MovieDto movieDto) {
        currentPort().save(movieDto);
    }
}

```

Как это работает:

- Spring контейнер создаёт оба бина (dbMoviePort и jsonMoviePort) при старте приложения;
- @Qualifier явно указывает, какой именно бин внедрить в каждое поле;

					МИВУ.09.03.02 009 ПЗ	Лист
						48
Изм.	Лист	№ докум.	Подпись	Дата		

- DataModeService получает текущий режим из HttpSession (устанавливается пользователем на странице /mode);
- Метод currentPort() возвращает нужный адаптер в зависимости от режима;
- Вся бизнес-логика (getAllMovies, addMovie и т.д.) работает с абстрактным портом, не зная о конкретной реализации.

Это решение обеспечивает соблюдение принципа Open/Closed Principle из SOLID: добавление нового хранилища (например, MongoDB) потребует лишь создания нового класса-адаптера и регистрации его в Spring-контексте — бизнес-логика останется неизменной.

Таблица 5 — Полный список реализованных адаптеров

Сущность	Порт	DB-адаптер	JSON-адаптер	JSON-файл
Movie	MoviePort	DbMovieAdapter	JsonMovieAdapter	movies.json
Series	SeriesPort	DbSeriesAdapter	JsonSeriesAdapter	series.json
Season	SeasonPort	DbSeasonAdapter	JsonSeasonAdapter	seasons.json
Episode	EpisodePort	DbEpisodeAdapter	JsonEpisodeAdapter	episodes.json
Actor	ActorPort	DbActorAdapter	JsonActorAdapter	actors.json
Genre	GenrePort	DbGenreAdapter	JsonGenreAdapter	genres.json
Director	DirectorPort	DbDirectorAdapter	JsonDirectorAdapter	directors.json
User	UserPort	DbUserAdapter	JsonUserAdapter	users.json

3.3 Реализация работы с мультимедиа-контентом

3.3.1. Раздача статических медиафайлов

Для воспроизведения видео через HTML5-плеер необходимо обеспечить доступ к видеофайлам, расположенным на диске (вне classpath приложения). Это реализуется через класс WebConfig, который регистрирует обработчик статических ресурсов:

```
package org.example.cinemahome.config;
```

```
import org.springframework.context.annotation.Configuration;
```

```

import
org.springframework.web.servlet.config.annotation.ResourceHandlerReg
istry;

import
org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addResourceHandlers(ResourceHandlerRegistry
registry) {
        String videoLocation = "file:/home/student/videos/";
        registry.addResourceHandler("/media/**")
            .addResourceLocations(videoLocation);
    }
}

```

После такой настройки любой файл из директории /home/student/videos/ становится доступен по URL-шаблону /media/{путь_к_файлу}. Например, файл /home/student/videos/интерstellar.mp4 будет доступен по адресу http://localhost:8080/media/интерstellar.mp4.

3.3.2. HTML5-видеоплеер для фильмов

Шаблон player/movie-player.html использует стандартный HTML5-тег <video> с атрибутом controls:

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title th:text="'Смотрим: ' + ${movie.title}">Плеер</title>
    <link rel="stylesheet" href="/css/style.css">
</head>
<body>
<div class="container">

```

```

<header>
    <h1 th:text="\${movie.title}">Фильм</h1>
    <nav>
        <a href="/">Главная</a>
        <a href="/movies">К списку фильмов</a>
        <a th:href="@{/movies/{id} (id=\${movie.id})}">К
описанию</a>
    </nav>
</header>
<main>
    <p th:text="'Путь к файлу: ' + \${movie.filePath}"></p>
    <video width="800" controls>
        <source th:src="@{'/media/' + \${movie.filePath}}"
            type="video/mp4">
        Ваш браузер не поддерживает воспроизведение видео.
    </video>
</main>
</div>
</body>
</html>

```

Атрибут `th:src` с использованием Thymeleaf-выражения `@{...}` автоматически формирует корректный URL с учётом контекста приложения.

3.3.3. Навигация по эпизодам сериала

Для сериалов реализована полноценная навигация между эпизодами, включающая три компонента:

- Кнопка «Предыдущая серия» — доступна, если текущий эпизод не является первым;
- Кнопка «Следующая серия» — доступна, если текущий эпизод не является последним;
- Панель навигации по всему сезону — визуальный ряд квадратных кнопок с номерами эпизодов.

```

@Controller
public class SeriesPlayerController {
    @Autowired private EpisodeService episodeService;
    @GetMapping("/player/series/{episodeId}")
    public String playEpisode(@PathVariable Long episodeId,
Model model) {
        EpisodeDto episode =
episodeService.getEpisodeById(episodeId);
        if (episode == null || episode.getFilePath() == null
            || episode.getFilePath().isBlank()) {
            return "error";
        }
        EpisodeDto next =
episodeService.getNextEpisode(episode);
        EpisodeDto prev =
episodeService.getPreviousEpisode(episode);
        List<EpisodeDto> seasonEpisodes =

episodeService.getEpisodesBySeasonId(episode.getSe
asonId());

        model.addAttribute("episode", episode);
        model.addAttribute("nextEpisode", next);
        model.addAttribute("prevEpisode", prev);
        model.addAttribute("seasonEpisodes", seasonEpisodes);
        return "player/series-player";
    }
}

```

Контроллер подготавливает для шаблона четыре атрибута: текущий эпизод, следующий, предыдущий и список всех эпизодов сезона.

```

<main>
    <video width="800" controls>
        <source th:src="@{'/media/' + ${episode.filePath}}"
type="video/mp4">

```

```

</video>
<!-- Навигация prev/next -->
<div style="margin-top: 2rem; display: flex; gap: 1rem;">
  <a th:if="{prevEpisode != null}"

  th:href="{@{/player/series/{id} (id={prevEpisode.id}) }}"
    class="btn-primary" style="background: #555;">
    ◀ Предыдущая серия
  </a>
  <span th:if="{prevEpisode == null}"
    style="visibility: hidden;">◀</span>
  <a th:if="{nextEpisode != null}"

  th:href="{@{/player/series/{id} (id={nextEpisode.id}) }}"
    class="btn-primary">
    Следующая серия ▶
  </a>
  <span th:if="{nextEpisode == null}">
    Это последняя серия сезона.
  </span>
</div>
<!-- Панель всех эпизодов сезона -->
<div style="margin-top: 2rem;">
  <h3>Все серии сезона</h3>
  <div style="display: flex; flex-wrap: wrap; gap:
0.5rem;">
    <a th:each="ep : {seasonEpisodes}"
      th:href="{@{/player/series/{id} (id={ep.id}) }}"
      th:text="{ep.episodeNumber}"
      th:classappend="{ep.id == episode.id} ?
'current-ep' : ''"
      class="episode-square">
    </a>
  </div>
</div>

```

```
</main>
```

```
.episode-square {
    display: inline-flex;
    justify-content: center;
    align-items: center;
    width: 40px;
    height: 40px;
    border: 1px solid #ccc;
    text-decoration: none;
    color: #333;
    font-weight: bold;
    background: var(--card);
    transition: all 0.2s;
}

.episode-square:hover { background: #f0f0f0; }

.current-ep {
    background-color: var(--accent) !important;
    color: #fff !important;
    border-color: var(--accent) !important;
}
```

Класс `current-ep` динамически добавляется через Thymeleaf-выражение `th:classappend`, выделяя текущий эпизод акцентным цветом.

3.3.4. Каскадное создание структуры сериала

При добавлении сериала через админ-панель система автоматически создаёт полную иерархическую структуру: сериал → сезон → эпизоды. Данная логика инкапсулирована в методе `saveSeriesWithStructure()` адаптеров `SeriesPort`.

```
@Override
public void saveSeriesWithStructure(SeriesDto dto) {
    // 1. Сохраняем сериал
    Series s = new Series();
    s.setTitle(dto.getTitle());
}
```



```

        s.setDescription(dto.getDescription());
        s.setStatus("released");
        s.setIsWatched(false);
        Integer seasonNumber = (dto.getSeasonNumber() != null)
            ? dto.getSeasonNumber() : 1;
        Integer episodesCount = (dto.getEpisodesCount() != null
            && dto.getEpisodesCount() > 0)
            ? dto.getEpisodesCount() : 1;
        s.setSeasonsCount(1);
        s.setEpisodesCount(episodesCount);
        Series savedSeries = seriesRepository.save(s);
        // 2. Создаём сезон
        Season season = new Season();
        season.setSeries(savedSeries);
        season.setSeasonNumber(seasonNumber);
        season.setEpisodesCount(episodesCount);
        season.setStatus("released");
        season.setTitle(seasonNumber + " сезон");
        Season savedSeason = seasonRepository.save(season);
        // 3. Создаём эпизоды по шаблону пути
        String folder = dto.getFolder();
        for (int epNum = 1; epNum <= episodesCount; epNum++) {
            Episode ep = new Episode();
            ep.setSeason(savedSeason);
            ep.setEpisodeNumber(epNum);
            ep.setTitle(epNum + " серия");
            ep.setFilePath(folder + "/" + seasonNumber
                + " сезон/" + epNum + " серия.mp4");
            episodeRepository.save(ep);
        }
    }
}

```

Путь к видеофайлу каждого эпизода формируется автоматически по шаблону {folder}/{seasonNumber} сезон/{epNum} серия.mp4, что соответствует типичной структуре локальных медиатек.

3.4 Реализация управления ролями и безопасности

3.4.1. Конфигурация Spring Security

@BeanБезопасность приложения реализована через класс SecurityConfig, который настраивает цепочку фильтров Spring Security:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/movies/**", "/", "/register", "/login",
                "/css/**", "/js/**", "/api/public/**",
                "/player/**", "/mode", "/access-denied"
            ).permitAll()
            .requestMatchers("/films/**", "/api/movies/**")
                .hasAnyRole("MODERATOR", "ADMIN")
            .requestMatchers("/admin/**", "/users/**",
"/api/admin/**")
                .hasAnyRole("MODERATOR", "ADMIN")
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login")
            .loginProcessingUrl("/login")
            .failureUrl("/login?error=true")
            .permitAll()
        )
        .logout(logout -> logout.permitAll())
        .exceptionHandling(ex -> ex
            .accessDeniedPage("/access-denied"))
    }
}
```

```

    );
    // Временно отключаем CSRF для упрощения POST-запросов
    http.csrf(csrf -> csrf.disable());
    return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}

```

Ключевые аспекты конфигурации:

- Публичные ресурсы (permitAll): главная страница, регистрация, вход, статика (CSS/JS), плеер, страница переключения режима и страница 403;
- Защищённые ресурсы: админ-панель и API-эндпоинты доступны только пользователям с ролями MODERATOR или ADMIN;
- Обработка ошибок входа: при неверном логине/пароле пользователь перенаправляется на /login?error=true;
- Обработка ошибок доступа: при попытке доступа к защищённому ресурсу без прав — редирект на /access-denied (HTTP 403);
- BCrypt — адаптивная функция хеширования паролей с настраиваемым коэффициентом стоимости.

3.4.2. Интеграция с базой данных пользователей

Для интеграции Spring Security с хранилищем пользователей реализован класс CustomUserDetailsService:

```

@Service
public class CustomUserDetailsService implements
UserDetailsService {
    @Autowired private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)

```

```

        throws UsernameNotFoundException {
            User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException(
                    "Пользователь не найден: " + username));
            return
org.springframework.security.core.userdetails.User.builder()
                .username(user.getUsername())
                .password(user.getPassword())
                .roles(user.getRoleName().toUpperCase())
                .build();
        }
    }
}

```

Метод `run()` выполняется один раз при старте приложения после полной инициализации Spring-контекста. Это гарантирует, что репозитории и сервисы уже готовы к работе.

Учётные данные по умолчанию:

- admin / admin (роль ADMIN) — полный доступ;
- moderator / moderator (роль moderator) — доступ к админ-панели.

3.4.4. Обработка ошибок доступа (HTTP 403)

Для централизованной обработки ситуаций, когда пользователь пытается получить доступ к защищённому ресурсу без соответствующих прав, реализован отдельный контроллер и шаблон:

```

@Controller
public class AccessDeniedController {
    @GetMapping("/access-denied")
    public String accessDenied() {
        return "access-denied";
    }
}

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

```

```

<head>
    <meta charset="UTF-8">
    <title>Доступ запрещен – CinemaHome</title>
    <link rel="stylesheet" href="/css/style.css">
</head>
<body>
<div class="container center">
    <h1>403 – Доступ запрещен</h1>
    <p style="font-size: 1.2rem; margin: 2rem 0; color:
#b71c1c;">
        У вас недостаточно прав для данного действия
    </p>
    <a href="/" class="btn-primary">На главную</a>
</div>
</body>
</html>

```

Конфигурация Spring Security через `exceptionHandling(ex -> ex.accessDeniedPage("/access-denied"))` автоматически перенаправляет пользователя на эту страницу при попытке несанкционированного доступа.

3.4.5. Декларативная проверка ролей через `@PreAuthorize`

Для более тонкого контроля доступа на уровне методов контроллеров используется аннотация `@PreAuthorize`:

```

@GetMapping("/admin/movies")
@PreAuthorize("hasRole('MODERATOR') or hasRole('ADMIN')")
public String adminMovies(Model model) {
    model.addAttribute("movies", movieService.getAllMovies());
    return "admin/form";
}

```

Данная аннотация обеспечивает двойной уровень защиты: сначала URL проверяется фильтром `SecurityFilterChain`, а затем — методом контроллера. Это

гарантирует безопасность даже при изменении URL-маппинга.

На уровне шаблонов Thymeleaf проверка ролей осуществляется через пространство имён sec::

```
<div sec:authorize="hasRole('MODERATOR') or hasRole('ADMIN')">
    <!-- Контент только для модераторов и администраторов -->
</div>
```

3.5 Реализация пользовательского интерфейса

3.5.1. Архитектура UI

Пользовательский интерфейс построен на Thymeleaf с минималистичной CSS-вёрсткой. Основные принципы дизайна:

- ~ Шрифт Inter — современный sans-serif шрифт с хорошей читаемостью;
- ~ Приглушённая цветовая палитра — фон #fafafa, карточки #ffffff, акцентный синий #0066cc;
- ~ Отсутствие скруглений и теней — чистый, минималистичный вид;
- ~ CSS-переменные (:root) — для единообразия и лёгкой смены темы;
- ~ CSS Grid — для адаптивной сетки карточек.

```
:root {
    --bg: #fafafa;
    --card: #ffffff;
    --text: #222222;
    --muted: #767676;
    --accent: #0066cc;
    --radius: 0;
    --shadow: none;
}
```

```
body {
    margin: 0;
```

```

font-family: "Inter", -apple-system, BlinkMacSystemFont,
              "Segoe UI", Roboto, Helvetica, Arial, sans-
serif;

background: var(--bg);
color: var(--text);
line-height: 1.5;
}

```

3.5.2. Структура шаблонов

Все Thymeleaf-шаблоны расположены в директории `src/main/resources/templates/` и организованы по функциональным группам:

Таблица 6 — Структура Thymeleaf-шаблонов

Директория	Шаблон	Назначение
/	index.html	Главная страница с рекомендацией
/	error.html	Страница 404
/	access-denied.html	Страница 403
/movie/	list.html	Список фильмов
/movie/	detail.html	Детальная страница фильма
/series/	list.html	Список сериалов
/series/	episodes.html	Список эпизодов сезона
/player/	movie-player.html	Плеер фильма
/player/	series-player.html	Плеер эпизода с навигацией
/admin/	form.html	Форма добавления фильма
/admin/	series-form.html	Форма добавления сериала
/admin/	genre-form.html	Форма управления жанрами
/admin/	actor-form.html	Форма управления актёрами
/admin/	director-form.html	Форма управления режиссёрами
/auth/	login.html	Форма входа
/auth/	register.html	Форма регистрации
/mode/	select.html	Выбор режима хранения
/actor/	list.html	Список актёров
/genre/	list.html	Список жанров

3.5.3. Главная страница и рекомендации

Главная страница (`index.html`) отображает «фильм дня» — случайно выбранный из каталога фильм с тегом настроения, сгенерированным MoodAnalyzer:

```

<main class="hero">
  <!-- Рекомендация "фильма дня" -->
  <div class="pick-card" th:if="${movie}">
    
    <div class="info">
      <h2 th:text="${movie.title}">Название</h2>
      <p class="mood"
        th:text="'Настроение: ' + ${movie.moodTag}"
        th:classappend="${movie.moodTag}">
        Настроение
      </p>
      <p class="desc"
th:text="${movie.description}">Описание</p>
      <a th:href="@{/player/movie/{id} (id=${movie.id})}"
        class="btn-primary">Смотреть</a>
    </div>
  </div>
  <!-- Заглушка при пустом каталоге -->
  <div class="empty" th:unless="${movie}">
    <h2>Фильмов пока нет</h2>
    <p>Зайдите в <a href="/admin/movies">панель адми-
на</a>
    и добавьте что-нибудь!</p>
  </div>
</main>

```

3.5.4. Панель управления контентом

На главной странице реализован блок быстрого перехода к админ-формам с выпадающим списком из 5 пунктов:

```

<div style="margin-top: 2rem; background: var(--card);
  border: 1px solid #e5e5e5; padding: 1.5rem;">
  <h3>Панель управления контентом</h3>

```

					МИВУ.09.03.02 009 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		62


```

        <div style="display: flex; gap: 1rem; align-items:
center;">

        <label for="addType">Добавить:</label>
        <select id="addType" name="type">
            <option value="movie">Фильм</option>
            <option value="series">Сериал</option>
            <option value="genre">Жанр</option>
            <option value="actor">Актёр</option>
            <option value="director">Режиссер</option>
        </select>
        <button type="button" class="btn-primary"
            onclick="goToAddPage()">Перейти</button>

    </div>
</div>

<script>
function goToAddPage() {
    const type = document.getElementById('addType').value;
    switch(type) {
        case 'movie':    window.location.href = '/admin/movies';
break;

                case 'series':    window.location.href =
'/admin/series'; break;
                case 'genre':    window.location.href =
'/admin/genres'; break;
                case 'actor':    window.location.href =
'/admin/actors'; break;
                case 'director': window.location.href =
'/admin/directors'; break;
                default:        window.location.href =
'/admin/movies';
    }
}
</script>

```

Использование конструкции switch вместо цепочки if-else обеспечивает более читаемый и поддерживаемый код.

3.5.5. Адаптивная сетка карточек

Для списков фильмов и сериалов используется CSS Grid с автоматическим расчётом количества колонок:

```
.grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(200px,
1fr));
    gap: 1.5rem;
}

.movie-card {
    background: var(--card);
    border: 1px solid #e5e5e5;
    padding: 1rem;
    text-align: center;
}
```

Функция auto-fill с minmax(200px, 1fr) автоматически подстраивает количество колонок под ширину экрана, обеспечивая адаптивность без медиа-запросов.

3.е.6. Цветовые метки настроения

Для визуального выделения тегов настроения (Calm, Romantic, Thrilling и т.д.) используются CSS-классы, динамически добавляемые через th:classappend:

```
.mood {
    display: inline-block;
    padding: .25rem .5rem;
    font-size: .8rem;
    background: #f2f2f2;
    color: var(--muted);
}

.mood.Romantic { background: #fce4ec; color: #880e4f; }
```

```
.mood.Thrilling    { background: #ffebee; color: #b71c1c; }
.mood.Calm        { background: #e8f5e9; color: #1b5e20; }
.mood.Nostalgic   { background: #f3e5f5; color: #4a148c; }
.mood.Chaotic     { background: #fff3e0; color: #e65100; }
.mood.Existential { background: #eceff1; color: #263238; }
```

Каждому настроению соответствует свой приглушённый цвет, что создаёт визуальный паттерн и облегчает восприятие каталога.

3.6 Реализация модуля рекомендаций и уведомлений

3.6.1. Сервис рекомендаций (CurationService)

Сервис CurationService отвечает за формирование персональной рекомендации «фильма дня» на главной странице. Его логика включает:

- Получение полного списка фильмов из репозитория;
- Присвоение каждому фильму тега настроения через MoodAnalyzer;
- Случайный выбор одного фильма из списка;
- Формирование объекта-заглушки при пустом каталоге.

```
@Service
public class CurationService {
    @Autowired private MovieRepository movieRepository;
    @Autowired private MoodAnalyzer moodAnalyzer;
    public MovieDto recommendForToday() {
        List<MovieDto> movies =
movieRepository.findAll().stream()
            .map(movie -> new MovieDto(
                movie.getId(),
                movie.getTitle(),
                movie.getDescription(),
                moodAnalyzer.analyzeMood(),
                movie.getFilePath(),
                movie.getGenres().stream()
```

```

        .map(g ->
g.getId()).collect(Collectors.toList()),
        movie.getActors().stream()
        .map(a ->
a.getId()).collect(Collectors.toList())
    ))
    .collect(Collectors.toList());
    if (movies.isEmpty()) {
        return new MovieDto(null, "No movies yet",
            "Add something to the catalog", "Bored",
            null, List.of(), List.of());
    }
    return movies.get(new Random().nextInt(movies.size()));
}
}

```

3.6.2. Генератор настроения (MoodAnalyzer)

MoodAnalyzer — простой компонент, который возвращает одно из шести настроений случайным образом:

```

@Component
public class MoodAnalyzer {
    private static final String[] MOODS = {
        "Calm", "Romantic", "Thrilling",
        "Nostalgic", "Chaotic", "Existential"
    };
    public String analyzeMood() {
        return MOODS[new Random().nextInt(MOODS.length)];
    }
}

```

В реальной системе данный компонент мог бы анализировать историю просмотров, время суток, день недели и другие контекстные данные для более точного подбора настроения. В рамках учебного проекта реализована упрощённая вер-

сия, достаточная для демонстрации архитектурного паттерна.

3.6.3. Система уведомлений (Observer)

Система уведомлений о новых релизах реализована через паттерн Observer. Интерфейс `ReleaseNotifier` определяет единый контракт для всех наблюдателей:

```
package org.example.cinemahome.observer;

public interface ReleaseNotifier {
    void sendNotification();
}
```

В проекте реализованы три наблюдателя, каждый из которых представляет отдельный канал уведомлений:

~ `EmailNotifier` (основной, `@Primary`);

```
@Component
@Primary
public class EmailNotifier implements ReleaseNotifier {
    @Override
    public void sendNotification() {
        System.out.println("Email sent: 'New movie alert! "
            + "(Probably just a demo) '");
    }
}
```

~ `InAppNotifier` — внутриприложенное уведомление;

```
@Component
public class InAppNotifier implements ReleaseNotifier {
    @Override
    public void sendNotification() {
        System.out.println("In-app notification: "
            + "'A movie was added. You didn't notice.'");
    }
}
```

```
}
```

~ MoodNotifier — персонализированное уведомление.

```
@Component
public class MoodNotifier implements ReleaseNotifier {
    @Autowired private MoodAnalyzer moodAnalyzer;

    @Override
    public void sendNotification() {
        System.out.println("Mood-based alert: "
            + "'This movie fits your current vibe: "
            + moodAnalyzer.analyzeMood() + "'");
    }
}
```

3.6.4. Сервис-оркестратор уведомлений

Класс ReleaseNotifierService получает список всех зарегистрированных наблюдателей через механизм Dependency Injection (@Autowired List<ReleaseNotifier>) и обходит его, вызывая sendNotification() у каждого:

```
@Service
public class ReleaseNotifierService {
    @Autowired private List<ReleaseNotifier> notifiers;

    public void notifyAboutRelease() {
        notifiers.forEach(ReleaseNotifier::sendNotification);
    }
}
```

Преимущества такого подхода:

~ Автоматическое обнаружение: Spring самостоятельно собирает все бины, реализующие ReleaseNotifier, в список.;

					МИВУ.09.03.02 009 ПЗ	Лист
						68
Изм.	Лист	№ докум.	Подпись	Дата		

~ Расширяемость: добавление нового канала уведомлений (Telegram-бот, SMS, push-уведомление) требует лишь создания нового класса с аннотацией `@Component` — без изменения существующего кода;

~ Соблюдение ОСР: система открыта для расширения, но закрыта для модификации.

3.7 Архитектурные особенности реализации

3.7.1 Многослойная архитектура

Приложение разделено на 5 логических слоёв, каждый из которых имеет строго определённую ответственность:

~ Controller (`@Controller`, `@RestController`) — обработка HTTP-запросов, подготовка модели для Thymeleaf, валидация входных данных;

~ Service (`@Service`) — инкапсуляция бизнес-логики, оркестрация портов, координация сложных операций;

~ Port (Java-интерфейсы) — контракты доступа к данным, обеспечивающие инверсию зависимостей;

~ Adapter (`@Component`) — конкретные реализации портов для различных хранилищ;

~ Repository (`JpaRepository`) — низкоуровневый доступ к данным в DB-режиме.

Поток данных в системе представлен на рисунке 6:

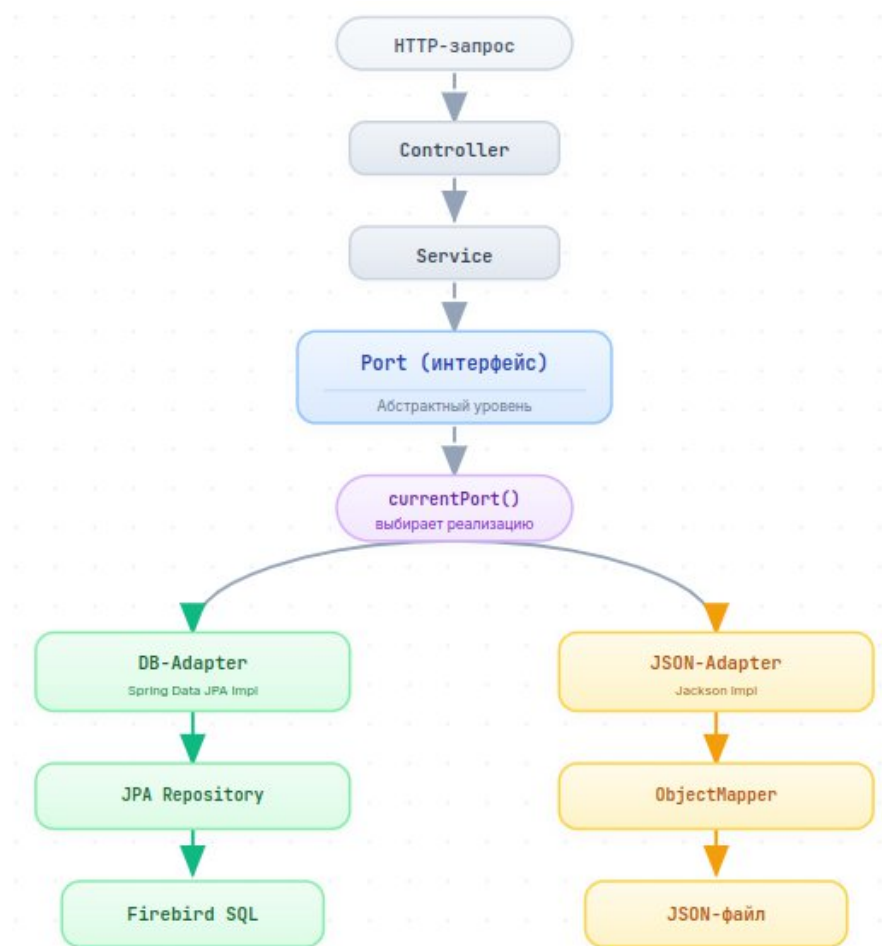


Рисунок 6 — Представление данных в системе

3.7.2. Dependency Injection и Inversion of Control

Все компоненты приложения получают зависимости через контейнер Spring IoC, что обеспечивает слабую связанность (low coupling) между классами. Используются следующие аннотации:

- ~ @Component — базовая аннотация для бинов (адаптеры, утилиты);
- ~ @Service — специализация для сервисного слоя;
- ~ @Repository — специализация для слоя доступа к данным;
- ~ @Controller — специализация для веб-слоя;
- ~ @Configuration — для классов конфигурации;
- ~ @Autowired — для внедрения зависимостей;
- ~ @Qualifier — для выбора конкретной реализации при наличии

нескольких кандидатов.

Ручное создание объектов через `new` в бизнес-коде полностью отсутствует — все объекты создаются и управляются Spring-контейнером. Это обеспечивает:

Тестируемость: зависимости можно легко заменить на mock-объекты.

Жизненный цикл: Spring управляет созданием и уничтожением бинов.

Ленивую инициализацию: бины создаются только при первом обращении.

3.7.3. Управление транзакциями

При сохранении сложных структур (сериал + сезон + эпизоды) применяется каскадное сохранение через JPA-аннотации:

```
@OneToMany(mappedBy = "series",
             cascade = CascadeType.ALL,
             orphanRemoval = true)
private List<Season> seasons;
```

`CascadeType.ALL` гарантирует, что при сохранении сериала автоматически сохраняются все его сезоны и эпизоды. `orphanRemoval = true` обеспечивает автоматическое удаление дочерних записей при их исключении из коллекции.

Для явного управления транзакциями в сервисном слое используется аннотация `@Transactional`:

```
@Transactional
public void addSeries(SeriesDto dto) {
    currentPort().saveSeriesWithStructure(dto);
}
```

Это гарантирует, что операция будет выполнена либо полностью, либо не выполнена вовсе (ACID-свойства).

3.7.4. Условная конфигурация бинов

Класс `DbConfig` использует аннотацию `@ConditionalOnMissingBean(DataSource.class)`, что предотвращает конфликт бинов `DataSource` при запуске тестов:

```

@Configuration
public class DbConfig {
    @Bean
    @ConditionalOnMissingBean (DataSource.class)
    public DataSource dataSource() {
        DriverManagerDataSource ds = new
DriverManagerDataSource ();
        ds.setDriverClassName ("org.firebirdsql.jdbc.FBDriver");
        ds.setUrl ("jdbc:firebirdsql://localhost:3050"
+ " //home/student/fb-data/cinema.fdb");
        ds.setUsername ("SYSDBA");
        ds.setPassword ("masterkey");
        return ds;
    }
}

```

3.7.5. API-документация (OpenAPI)

Для автоматической генерации документации REST API используется библиотека `springdoc-openapi 2.3.0`. Конфигурация задаётся в классе `OpenApiConfig`:

```

@Configuration
public class OpenApiConfig {
    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("API CinemaHome")
                .version("1.0")
                .description("API для управления кинотеатром,
"
+ "сеансами и бронированием билетов")
                .license(new License()
                    .name("Apache 2.0")

                .url("https://www.apache.org/licenses/LICENSE-2.0"))
    }
}

```

```

        );
    }
}

```

После запуска приложения интерактивная документация становится доступна по адресам:

- Swagger UI: <http://localhost:8080/swagger-ui.html>;
- OpenAPI JSON: <http://localhost:8080/api-docs>.

Swagger UI автоматически сканирует все `@RestController` и `@GetMapping/@PostMapping`-аннотации, формируя интерактивную документацию с возможностью отправки тестовых запросов прямо из браузера.

3.7.6. Логирование и отладка

Для диагностики работы приложения настроены различные уровни логирования в `application.properties`:

```

logging.level.org.springframework.web=DEBUG
# logging.level.org.thymeleaf=DEBUG
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

DEBUG для Spring Web: логирует все входящие HTTP-запросы, маппинг контроллеров и обработку исключений.

- `show-sql`: выводит все SQL-запросы, генерируемые Hibernate, в консоль;
- `format_sql`: форматирует SQL-запросы для удобочитаемости.

4 Тестирование

Одним из важнейших этапов разработки программного продукта является его тестирование и отладка. Тестирование позволяет выявить скрытые и явные дефекты программы, а также подтвердить её пригодность для практического применения. В рамках курсового проекта была применена комбинированная стратегия тестирования, включающая как ручную верификацию пользовательских сценариев, так и автоматизированное модульное и интеграционное тестирование.

Цель тестирования — проверить корректность работы всех ключевых подсистем приложения CinemaHome: механизма динамической смены хранилища (Port-Adapter), модуля рекомендаций, системы уведомлений (Observer), а также пользовательского интерфейса и ролевой модели безопасности.

4.1 Цели и стратегия тестирования

Стратегия тестирования была выстроена по принципу «от пользовательского опыта — к внутренним компонентам». Такой подход обусловлен спецификой разрабатываемого приложения: CinemaHome является пользовательским веб-сервисом, где именно поведение системы с точки зрения конечного пользователя (администратора, модератора или зрителя) является первичным критерием качества.

В связи с этим в проекте были выделены три уровня тестирования, применяемых в следующей последовательности:

Ручное (функциональное) тестирование — верификация пользовательских сценариев через браузер. Позволяет оценить систему целиком: от отрисовки HTML-шаблонов до взаимодействия с хранилищем данных и системой безопасности.

Автоматизированное интеграционное тестирование — проверка корректной загрузки Spring-контекста приложения и взаимодействия его компонентов.

Автоматизированное модульное (unit) тестирование — изолированная проверка отдельных классов и методов с использованием мокирования зависимостей.

Ручное тестирование было проведено в первую очередь, поскольку оно позволяет обнаружить дефекты, не выявляемые на уровне кода (например, проблемы вёрстки, некорректные редиректы, ошибки в шаблонах Thymeleaf). Только после подтверждения работоспособности пользовательских сценариев были написаны автоматизированные тесты для критических бизнес-компонентов.

4.2 Ручное (функциональное) тестирование

Ручное функциональное тестирование проводилось в браузере Mozilla Firefox на локальном развёртывании приложения по адресу <http://localhost:8080>. Тестирование охватило все основные пользовательские сценарии: аутентификацию, работу с ролями, управление контентом, переключение режимов хранения, воспроизведение видео и работу модуля рекомендаций.

4.2.1. Сценарии функционального тестирования

Все проверенные сценарии сведены в таблицу 7.

№	Модуль	Действие	Ожидаемый результат	Статус
1.	Аутентификация	Регистрация нового пользователя	Учётная запись создана, редирект на /login?registered	Выполнено
2.	Аутентификация	Регистрация с неполными данными	HTML5-валидация блокирует отправку формы	Выполнено
3.	Аутентификация	Вход с корректными данными (admin/admin)	Установлена сессия, редирект на /	Выполнено
4.	Аутентификация	Вход с неверным паролем	Редирект на /login?error=true , отображение сообщения	Выполнено
5.	Роли	Доступ к /admin/movies без роли MODERATOR	HTTP 403, редирект на /access-denied	Выполнено
6.	Роли	Доступ к /admin/movies с ролью ADMIN	Отображение админ-панели	Выполнено
7.	Роли	Доступ к /admin/genres с ролью moderator	Отображение формы управления жанрами	Выполнено

8.	Фильмы (DB)	Добавление фильма через панель админа	Фильм сохранён в Firebird, отображается в списке	Выполнено
9.	Фильмы (JSON)	Добавление фильма в режиме JSON	Запись в movies.json	Выполнено
10.	Сериалы	Добавление сериала (1 сезон, 10 эпизодов)	Каскадно созданы Series + Season + 10 Episode	Выполнено
11.	Переключение режима	POST /mode с параметром mode=JSON	Режим сохранён в HttpSession, адаптеры переключены	Выполнено
12.	Плеер (фильм)	Открытие /player/movie/{id}	Отображение HTML5-плеера с src /media/...	Выполнено
13.	Плеер (сериал)	Переход к следующей серии	Кнопка «Следующая серия» ведёт на следующий эпизод	Выполнено
14.	Плеер (сериал)	Переход к предыдущей серии	Кнопка «Предыдущая серия» ведёт на предыдущий эпизод	Выполнено
15.	Рекомендации	GET / с пустым каталогом	Отображается блок «Фильмов пока нет»	Выполнено
16.	Рекомендации	GET / с заполненным каталогом	Отображается случайный фильм с тегом настроения	Выполнено
17.	API-документация	GET /swagger-ui.html	Открыта страница Swagger UI	Выполнено
18.	Observer	Создание нового фильма через админ-панель	В консоли: 3 уведомления (Email, InApp, Mood)	Выполнено
19.	Справочники	Добавление жанра/актёра/режиссёра	Запись сохраняется в соответствующем хранилище	Выполнено
20.	Ошибки	Переход по несуществующему URL	Страница 404 «Сцена не найдена»	Выполнено

Ниже приведено детальное описание наиболее значимых сценариев.

4.2.2. Регистрация с некорректными данными

Для проверки надёжности системы валидации был проведён тест на регистрацию с заведомо неполными данными. Браузерная валидация HTML5-атрибута required и серверная валидация Spring заблокировали отправку формы. Результат представлен на рисунке 7.

Регистрация

...

 Заполните это поле.

Создать аккаунт

Уже есть аккаунт? [Войти](#)

Рисунок 7 — Попытка регистрации с некорректными данными

4.2.3. Авторизация с некорректными данными

Была предпринята попытка входа под несуществующей учётной записью. Spring Security корректно обработал ситуацию: CustomUserDetailsService выбросил UsernameNotFoundException, и на странице /login?error=true было отображено красное сообщение «Неверный логин или пароль» через Thymeleaf-условие th:if="\$ {param.error} ". Результат перехвата ошибки сервером показан на рисунке 8.

Рисунок 8 — Попытка авторизации с некорректными данными

4.2.4. Тестирование ролевой модели и страницы 403

Попытка доступа к админ-панели (/admin/movies) пользователем с ролью user блокируется фильтром Spring Security с HTTP-кодом 403 Forbidden и перенаправлением на страницу /access-denied. Это обеспечивается конфигурацией `exceptionHandling(ex -> ex.accessDeniedPage("/access-denied"))` в классе `SecurityConfig` и отдельным контроллером `AccessDeniedController`. Авторизованный пользователь с ролью ADMIN или MODERATOR успешно получает доступ к формам добавления контента. Результаты представлены на рисунках 9 и 10.

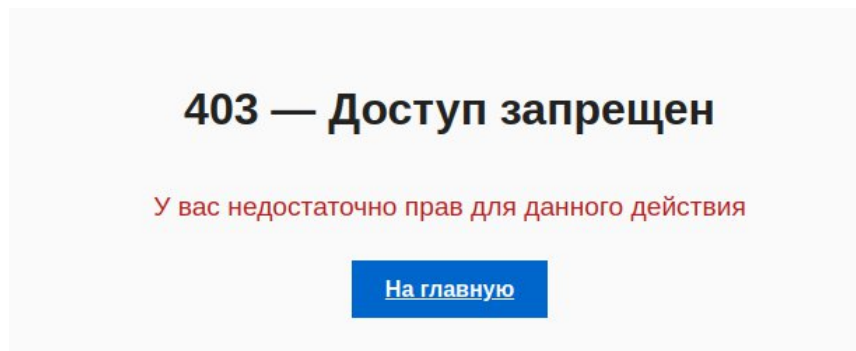


Рисунок 9 — Отказ в доступе (HTTP 403) с редиректом на /access-denied

Рисунок 10 — Успешный доступ к админ-панели

4.2.5. Переключение режима хранения (DB ↔ JSON)

Ключевой сценарий проекта — динамическая смена хранилища. При переходе на страницу /mode и выборе опции «JSON файлы» с последующим POST-запросом, DataModeController сохраняет значение DataMode.JSON в HttpSession.

Все последующие обращения к сервисам (MovieService, SeriesService, DirectorService и др.) через метод currentPort() возвращают соответствующие Json*Adapter. Добавленный в этом режиме фильм сохраняется в файл /home/student/cinema-json/movies.json. Результат представлен на рисунке 11.

localhost:8080/mode

Выберите источник данных

☒ База данных (Firebird)

☐ JSON файлы

Продолжить

Рисунок 11 — Страница переключения режима хранения

4.2.6. Воспроизведение видео через HTML5-плеер

При переходе на /player/movie/{id} контроллер PlayerController загружает объект MovieDto и передаёт его в шаблон player/movie-player.html. Thymeleaf формирует HTML5-тег <video> с источником /media/{filePath}, а WebConfig раздаёт файл из директории file:/home/student/videos/. Результат представлен на рисунке 12.

Путь к файлу: Tri_bogatirya.mp4



Рисунок 12 — Встроенный HTML5-видеоплеер

4.2.7. Навигация по эпизодам сериала

При просмотре эпизода сериала через `/player/series/{episodeId}` пользователю доступны:

кнопка « Предыдущая серия» (скрывается для первого эпизода);

кнопка «Следующая серия » (скрывается для последнего эпизода);

визуальная панель со всеми эпизодами текущего сезона, где текущий эпизод выделен акцентным цветом (CSS-класс `.current-ep`).

Данная логика реализуется через методы `findPrevious()` и `findNext()` в `EpisodePort` и поддерживается обоими адаптерами (DB через `findBySeason_IdAndEpisodeNumber`, JSON через Stream API с фильтрацией). Результат представлен на рисунке 13.

Путь к файлу: /media/жуки/4 сезон/4 серия.mp4

[◀ Предыдущая серия](#)[Следующая серия ▶](#)

Все серии сезона



Рисунок 13 — Плеер сериала с навигацией по эпизодам

4.2.8. Панель управления контентом

На главной странице присутствует блок быстрого перехода к админ-формам с выпадающим списком из 5 пунктов: Фильм, Сериал, Жанр, Актёр, Режиссёр. JavaScript-обработчик `goToAddPage()` использует конструкцию `switch` для навигации к соответствующему URL (`/admin/movies`, `/admin/series`, `/admin/genres`, `/admin/actors`). Результат представлен на рисунке 14.

Три богатыря и принцесса египта

Настроение: Nostalgic

В новогоднюю ночь, в то время, как Князь готовится произнести торжественную речь, а Юлий распаковывает подарки и следит за приготовлением праздничного стола, богатыри едут в гости к Алёше и встречают там странного товарища по имени Дурило. Оказывается, это тринадцатый месяц, который никому особенно не нужен, и который пытается осуществить своё заветное желание – стать самым главным месяцем в году. Только и надо ему, что притвориться Дедом Морозом, показать доверчивому Алёше пару фокусов, подружиться с богатырями и оказаться в Египте – стране фараонов и древних пирамид...

Смотреть

Панель управления контентом

Добавить:

Фильм

Перейти

Для доступа ко всем разделам нужно войти как модератор или админ.

Рисунок 14 — Панель управления контентом на главной странице

4.2.9. API-документация Swagger UI

Библиотека `springdoc-openapi-starter-webmvc-ui` версии 2.3.0 автоматически генерирует интерактивную документацию по REST API. При переходе на `/swagger-ui.html` открывается страница с полным описанием эндпоинтов (RecommendationController, публичные ресурсы). Результат представлен на рисунке 15.

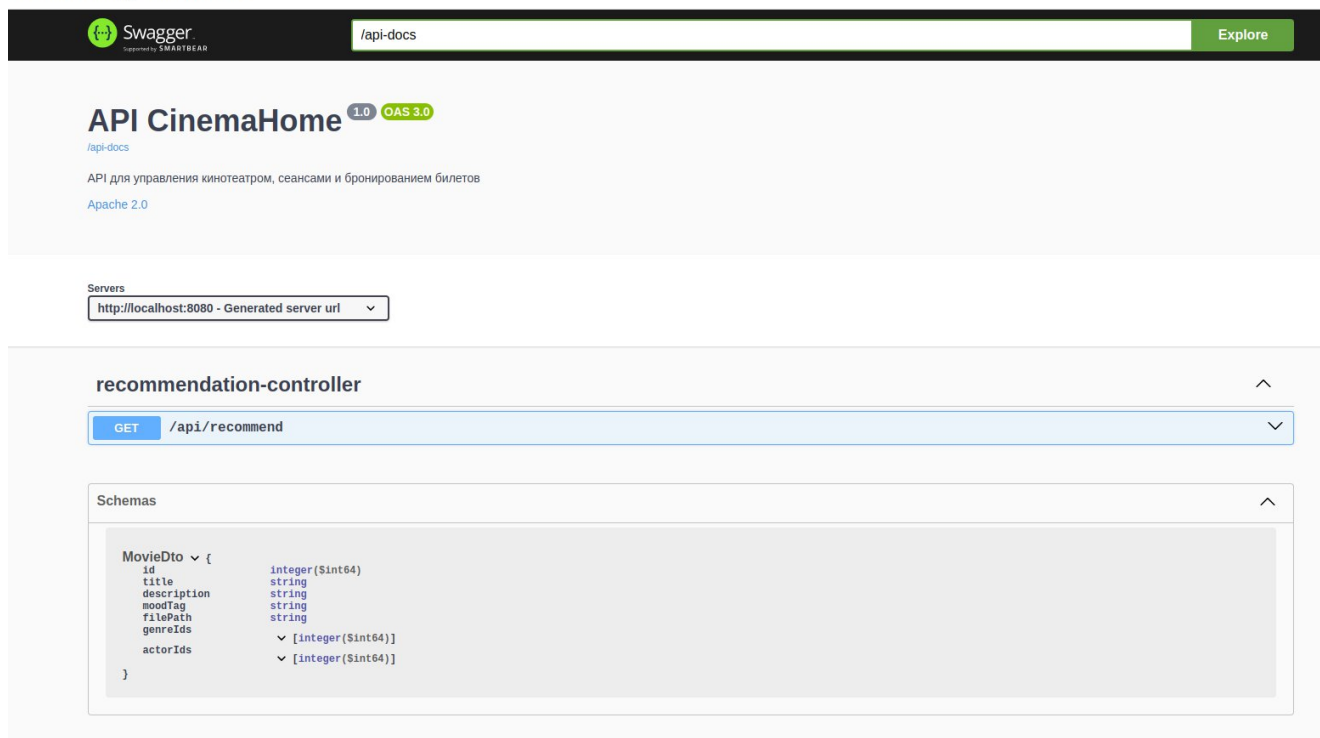


Рисунок 15 — Swagger UI

4.3 Инструментарий автоматизированного тестирования

После подтверждения работоспособности всех пользовательских сценариев через ручное тестирование, были написаны автоматизированные тесты для наиболее критичных с точки зрения бизнес-логики модулей. Для их реализации был выбран следующий набор библиотек, подключённых через build.gradle. Каждая библиотека решает свою специфическую задачу и работает в связке с остальными.

Таблица 8 — Инструментарий автоматизированных тестов

Инструмент	Версия	Назначение
JUnit 5 (Jupiter)	5.x (через spring-boot-starter-test)	Основной фреймворк для написания и запуска тестов
Mockito	5.+	Создание мок-объектов для изоляции тестируемых компонентов
AssertJ	3.x (через spring-boot-starter-test)	Fluent-стиль утверждений для повышения читаемости тестов

Spring Boot Test	3.4.0	Интеграционные тесты с загрузкой полного контекста
ReflectionTestUtils	Spring Test	Утилита для инъекции приватных полей в тестируемые объекты
Gradle Test Task	8.5	Автоматический прогон тестов и генерация HTML-отчётов

Ниже приведено подробное описание каждого инструмента: его назначения, принципа работы и особенностей применения в проекте.

4.3.1. JUnit 5 (Jupiter) — основа тестового фреймворка

JUnit 5 — современный стандарт модульного тестирования в Java-экосистеме, пришедший на смену JUnit 4. В отличие от предшественника, JUnit 5 имеет модульную архитектуру, состоящую из трёх подпроектов:

- JUnit Platform — платформа для запуска тестов, предоставляющая API для интеграции с IDE и системами сборки;
- JUnit Jupiter — новая программная модель для написания тестов с использованием аннотаций `@Test`, `@BeforeEach`, `@AfterEach`, `@ParameterizedTest`;
- JUnit Vintage — движок для обратной совместимости с тестами JUnit 3 и JUnit 4 (в проекте не используется).

Принцип работы: каждый метод, помеченный аннотацией `@Test`, рассматривается платформой как отдельный тестовый сценарий. JUnit Jupiter выполняет тесты в изолированных экземплярах тестового класса (по умолчанию создаётся новый экземпляр на каждый тест), что исключает побочные эффекты между тестами.

Аннотация `@BeforeEach` используется для подготовки тестового окружения перед каждым тестом (создание mock-объектов, инъекция зависимостей):

```
@BeforeEach
void setUp() {
    movieRepository = Mockito.mock(MovieRepository.class);
    moodAnalyzer = Mockito.mock(MoodAnalyzer.class);
    curationService = new CurationService();
    injectField(curationService, "movieRepository",
movieRepository);
}
```

```

        injectField(curationService, "moodAnalyzer", moodAnalyzer);
    }

```

JUnit 5 подключается в проект транзитивно через стартер spring-boot-starter-test и активируется в Gradle директивой `test { useJUnitPlatform() }`.

4.3.2. Mockito — мокирование зависимостей

Mockito — самый популярный фреймворк для создания mock-объектов (подставных объектов) в Java. Mock-объекты имитируют поведение реальных зависимостей (репозиториев, сервисов, внешних API) без обращения к ним, что позволяет тестировать классы изолированно.

Принцип работы: Mockito использует динамическую генерацию байт-кода (через библиотеку ByteBuddy) для создания прокси-классов, реализующих нужный интерфейс или наследующих нужный класс. Поведение mock-объекта настраивается через fluent API:

```

// Создание mock-объекта
MoodAnalyzer analyzer = mock(MoodAnalyzer.class);

// Настройка поведения: при вызове analyzeMood() вернуть "Calm"
when(analyzer.analyzeMood()).thenReturn("Calm");

// Проверка: был ли метод analyzeMood() вызван ровно 1 раз
verify(analyzer, times(1)).analyzeMood();

```

Ключевые возможности, использованные в проекте:

- `mock(Class)` — создание mock-объекта для заданного класса или интерфейса;
- `when(...).thenReturn(...)` — настройка возвращаемого значения при вызове метода;
- `verify(..., times(N))` — проверка количества вызовов метода (важно для тестирования паттерна Observer).

Применение в проекте: Mockito используется во всех unit-тестах (MoodNotifierTest, CurationServiceTest) для изоляции тестируемых компонентов от их зависимостей. Например, в CurationServiceTest создаются mock-объекты MovieRepository и MoodAnalyzer, что позволяет протестировать логику сервиса без обращения к реальной базе данных или генератору случайных чисел.

Особенность конфигурации: в build.gradle Mockito подключён как testImplementation 'org.mockito:mockito-core:5.+', что обеспечивает использование актуальной версии 5.x с поддержкой inline-mock-maker (возможность мокать final-классы и статические методы).

4.3.3. AssertJ — читаемые утверждения

AssertJ — библиотека для написания утверждений (assertions) в стиле fluent API. В отличие от стандартных методов JUnit (assertEquals, assertTrue), AssertJ предлагает более выразительный синтаксис, который читается как английский текст.

Принцип работы: все утверждения строятся через фабричный метод assertThat(actual), который возвращает объект-обёртку с цепочкой методов проверки:

```
// Стандартный JUnit
assertEquals(1, all.size());
assertEquals("Test movie", saved.getTitle());

// AssertJ (читается как предложение)
assertThat(all).hasSize(1);
assertThat(saved.getTitle()).isEqualTo("Test movie");
assertThat(byId).isNotNull();
```

Ключевые возможности, использованные в проекте:

- hasSize(N) — проверка размера коллекции;
- isEqualTo(...) — проверка равенства значений;
- isNotNull() / isNull() — проверка на null;

— Специализированные утверждения для коллекций, строк, дат, исключений.

Применение в проекте: AssertJ используется во всех unit-тестах как основной инструмент проверки результатов. Подключается транзитивно через spring-boot-starter-test.

4.3.4. Spring Boot Test — интеграционное тестирование

Spring Boot Test — модуль фреймворка Spring, предоставляющий специализированные аннотации и утилиты для тестирования Spring-приложений. Ключевой аннотацией является `@SpringBootTest`, которая поднимает полный Spring-контекст приложения, включая все бины, конфигурации, сервисы и репозитории.

Принцип работы: при запуске теста с `@SpringBootTest` Spring Boot выполняет те же шаги инициализации, что и при старте реального приложения:

- Сканирование классов с аннотациями `@Component`, `@Service`, `@Repository`, `@Controller`, `@Configuration`;
- Создание бинов и внедрение зависимостей;
- Инициализация `DataSource`, `EntityManagerFactory`, `SecurityFilterChain`;
- Запуск `CommandLineRunner` (в проекте — `AdminInitializer`).

Если на любом из этих этапов возникает ошибка (например, конфликт бинов, отсутствующая зависимость), тест завершается с ошибкой, что позволяет обнаруживать проблемы конфигурации на ранней стадии.

4.3.5. ReflectionTestUtils — инъекция приватных полей

ReflectionTestUtils — утилита из модуля spring-test, предоставляющая методы для работы с приватными полями и методами через рефлексия. Она является стандартом де-факто для инъекции зависимостей в unit-тестах, где не используется Spring-контекст.

Принцип работы: В unit-тестах мы не поднимаем Spring-контекст (это медленно и избыточно), а создаём тестируемые объекты через `new`. Однако эти объекты часто имеют приватные поля, помеченные `@Autowired`, которые в production-режиме заполняются контейнером Spring.

ReflectionTestUtils.setField(...) позволяет установить значение такого поля вручную:

```
JsonMovieAdapter adapter = new JsonMovieAdapter();  
// Подменяем приватную константу MOVIES_JSON_PATH на временный  
файл  
ReflectionTestUtils.setField(adapter, "MOVIES_JSON_PATH",  
                               tmp.getAbsolutePath());
```

Применение в проекте: В тесте JsonMovieRepositoryTest эта утилита используется для подмены пути к JSON-файлу, что позволяет тестировать адаптер на временном файле, не затрагивая рабочее хранилище /home/student/cinema-json/movies.json. Это обеспечивает изолированность тестов и их повторяемость.

В тестах MoodNotifierTest и CurationServiceTest используется собственный вспомогательный метод injectField(...), реализующий аналогичную логику через Java Reflection API.

4.3.7. Gradle Test Task — автоматизация запуска тестов

Gradle Test Task — встроенная задача системы сборки Gradle, автоматически обнаруживающая тестовые классы и выполняющая их. Результатом работы является HTML-отчёт, доступный по пути build/reports/tests/test/index.html.

Принцип работы:

- Gradle сканирует директорию src/test/java/ на наличие классов с методами @Test;
- Для каждого найденного класса создаётся отдельный тестовый runner;
- Выполняются все тестовые методы, результаты (успех/падение/время) фиксируются;
- Генерируется HTML-отчёт с детализацией по пакетам, классам и методам.

Применение в проекте: Запуск всех тестов осуществляется командой ./gradlew test. Эта же команда выполняется в GitHub Actions при каждом

push в ветку main, что обеспечивает непрерывную интеграцию: если тесты падают, PR не может быть смержен.

					МИВУ.09.03.02 009 ПЗ	Лист
						90
Изм.	Лист	№ докум.	Подпись	Дата		

Заключение

В ходе выполнения курсового проекта было разработано веб-приложение CinemaHome — адаптивная платформа локального медиаконтента с динамической сменой хранилища. Основной целью проекта являлось создание многофункционального приложения, способного гибко управлять каталогом фильмов и сериалов, воспроизводить видеофайлы через встроенный HTML5-плеер и прозрачно переключаться между двумя независимыми хранилищами данных — реляционной СУБД Firebird и файловым хранилищем в формате JSON.

В процессе разработки была спроектирована гексагональная архитектура (Port-Adapter) с элементами Layered Architecture, что обеспечило чёткое разделение ответственности между слоями Controller, Service, Port, Adapter и Repository. Такое решение позволило добиться слабой связанности модулей и соответствия принципам SOLID, в частности Dependency Inversion Principle и Open/Closed Principle.

В рамках работы были реализованы:

- поддержка семи типов сущностей (Movie, Series, Season, Episode, Actor, Genre, Director, User);
- два параллельных набора адаптеров (Db*Adapter и Json*Adapter) для каждого порта;
- паттерн Strategy для динамического выбора хранилища через DataModeService;
- паттерн Observer для системы уведомлений о релизах (EmailNotifier, InAppNotifier, MoodNotifier);
- ролевая модель безопасности (ADMIN, moderator, user) на базе Spring Security и BCrypt;
- пользовательский интерфейс на Thymeleaf с минималистичной CSS-вёрсткой;
- модуль рекомендаций (CurationService + MoodAnalyzer);

- автоматическое создание администратора и модератора при старте (AdminInitializer);
- API-документация через springdoc-openapi (Swagger UI).

В качестве технологического стека были выбраны Java 21, Spring Boot 3.4, Hibernate 6.6.2 с Community Dialects для Firebird, Jackson Databind 2.17.0, Thymeleaf, Spring Security, Lombok 1.18.36, сборщик Gradle 8.5.

Корректность работы ключевых модулей подтверждена модульными тестами (JUnit 5 + Mockito + AssertJ). Все 5 тестов проходят успешно (success rate 100 %).

В результате выполнения курсового проекта была создана полнофункциональная и масштабируемая платформа локального медиаконтента, соответствующая всем требованиям технического задания. Разработанный продукт наглядно демонстрирует применение принципов объектно-ориентированного проектирования, современных архитектурных паттернов и промышленных стандартов веб-разработки на платформе Java.

Список использованных источников

1 Ермаков, А. В. Разработка приложений на языке JAVA : учебное пособие / А. В. Ермаков, М. С. Королёв, А. К. Кузьмин. — Саратов : Саратовский государственный технический университет имени Ю.А. Гагарина, ЭБС АСВ, 2024. — 128 с. — ISBN 978-5-7433-3635-7.

2 Лазицкас, Е. А. Базы данных и системы управления базами данных [Электронный ресурс] : учебное пособие / Е. А. Лазицкас, И. Н. Загумённикова, П. Г. Гилевский. — Минск : РИПО, 2018. — 268 с. — Режим доступа: <http://www.iprbookshop.ru/93382.html>.

3 Мирошников, А. И. Архитектура систем управления базами данных [Электронный ресурс] : учебное пособие / А. И. Мирошников. — Липецк : ЛГТУ, ЭБС АСВ, 2018. — 94 с. — Режим доступа: <http://www.iprbookshop.ru/83189.html>.

4 Суханов, В. И. Разработка веб-приложений на платформе Spring : учебно-методическое пособие / В. И. Суханов ; под ред. С. И. Тимошенко. — Екатеринбург : Изд-во Уральского ун-та, 2023. — 180 с. — Режим доступа: <https://www.iprbookshop.ru/157380.html>.

5 Официальная документация Spring Boot 3.4 [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/3.4.0/reference/html/>